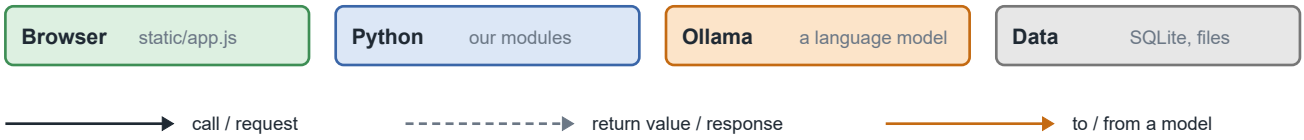
AI Invoice Assistant

Dataflow in scenarios – how the parts pass data to each other

For a reader who has never opened the code. Each scenario follows one user action from the click in the browser through the Python modules, the language models and the database, and back. The first part explains the pieces and the four “engines” every scenario reuses; the second part walks through 19 scenarios; the appendix lists where state lives and which function appears where.

Based on	commit f98254d (main), 21/10/2026
Code covered	app/api.py, app/chat/*.py, app/entities/*.py, app/core/*.py, static/app.js, modelfiles/*
Not covered	tests/, the Modelfile examples in detail, open problems (see docs/open_problems.md)

How to read the diagrams



1 = the row with the same number in the step table under the diagram

Text that a language model writes is shown as *e.g. “…”*: the model may word it differently. Text that Python writes is quoted exactly as it is in the code. Names such as Lena Berg, Karl Becker or Desk Lamp and numbers such as CUS-013 are examples.

Contents

Contents . . . 2

1 The big picture . . . 4

1.1 Three rules that shape every flow . . . 4

1.2 The parts and how they connect . . . 4

1.3 Who does what . . . 5

1.4 The stored data . . . 5

1.5 The objects that travel . . . 6

2 The engines every scenario reuses . . . 7

2.1 Conversation memory . . . 7

2.2 router.converse – which bot gets the message . . . 8

2.3 bot.converse – the change bots and their guards . . . 9

2.4 assistant.converse – the question bot with tools . . . 10

2.5 Saving – the WRITE endpoints . . . 11

2.6 The form contract . . . 11

3 Scenarios . . . 12

S1 Open the page and look at the tables . . . 13

S2 Create a customer in one message . . . 14

S3 Create a customer, the address is missing . . . 16

S4 Change a product's price with the edit form . . . 18

S5 A negative price is stopped before any bot . . . 20

S6 Delete a customer who still has invoices . . . 21

S7 A name matches two customers → chooser . . . 23

S8 The bot invents the record → Python asks . . . 25

S9 Create an invoice step by step . . . 27

S10 Mark an invoice as paid . . . 29

S11 Ask a question: where does a customer live? . . . 31

S12 List unpaid invoices / is one invoice paid? . . . 33

S13 Turnover per customer . . . 34

S14 Monthly turnover and turnover per product . . . 36

S15 A change through the assistant . . . 37

S16 Yes, but the data changed in the meantime . . . 39

S17 The Help button . . . 40

S18 A message that is not about invoicing . . . 41

S19 Other entry points . . . 42

Appendix A Where every piece of state lives 43

A.2 Which parts may change the database 43

Appendix B Function index 44

B.2 Words used in this document 44

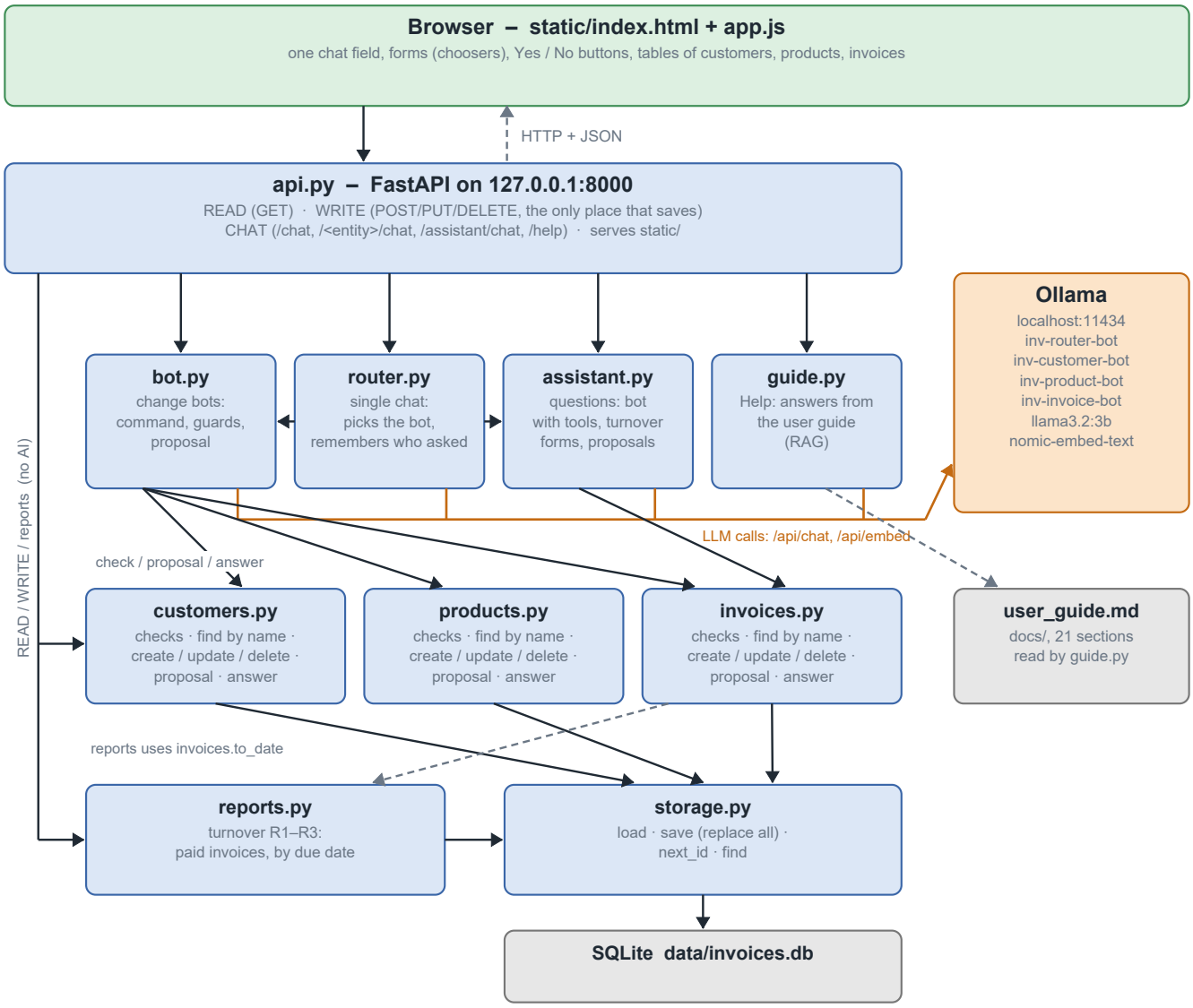
1 The big picture

The program is a small invoicing system that runs entirely on one computer: a web page, a Python web server (FastAPI) and local language models (Ollama, model llama3.2:3b). The user types into **one chat field**; the program understands the sentence, asks for what is missing through small forms, and finally offers a change with **Yes / No**. Tables on the right show the stored records.

1.1 Three rules that shape every flow

Rule	What it means for the dataflow
The models only understand language.	A model turns a sentence into a route (“customer”), a command line (CREATE_CUSTOMER {…}) or a tool call. Python then looks records up, checks the values, calculates totals and turnover, and writes most lists itself.
The chat never saves.	Every chat path ends in either a *reply* (text) or a *proposal* – a description of a REST call. Only when the user clicks Yes does the browser send that REST call to a WRITE endpoint, which checks the data a second time and saves it.
Forms speak in chat messages.	A form in the browser does not call a special endpoint. It builds an ordinary chat message in a fixed wording (e.g. “customer: Karl Becker”) and sends it to /chat like typed text. Bots and Python know these wordings.

1.2 The parts and how they connect



Read top to bottom: every request enters through api.py. Chat requests go down into the chat layer, which talks to Ollama for language and to the entity modules for facts. WRITE and READ requests skip the chat layer completely (the lane on the left): no model is involved in saving or in listing data.

1.3 Who does what

Part (file)	Job	Gets data from	Gives data to
static/app.js (+ index.html)	Chat field, Help button, forms, proposal with Yes/No, tables.	The user; JSON answers from the server.	POST /chat, POST /help, the WRITE endpoints after Yes, GET lists.
app/api.py	The entry point: one function per URL. Checks nothing itself; calls the modules and turns their errors into HTTP codes (400, 404, 422).	The browser (or any HTTP client).	router, bot, assistant, guide, the entity modules, reports.
app/chat/router.py	The single chat: asks the router model which bot a message belongs to, remembers which bot is waiting for an answer, adds the *ui* keys that open forms.	api.chat	bot.converse or assistant.converse
app/chat/bot.py	Talks to Ollama (chat). For the three change bots: reads the command line from the model's reply, runs the guards, asks the entity module to check and to build the proposal. Keeps all conversations (CONVERSATIONS).	router or api	Ollama; the check / proposal / answer functions of an entity module
app/chat/assistant.py	The question bot: a model with tools. Python answers read tools, shows forms, answers the turnover forms itself and turns write tools into proposals.	router or api	Ollama, storage, reports, the entity check functions
app/chat/guide.py	Help: finds the 3 sections of docs/user_guide.md closest in meaning to the question and lets a model answer only from them.	api.help_chat	Ollama (embed + chat)
app/entities/customers.py, products.py, invoices.py	Everything about one kind of record: checks, find by name, create / update / delete, the proposal and the text answer for the chat.	bot.py, assistant.py, api.py	storage
app/core/reports.py	Turnover R1 (per customer), R2 (per product), R3 (per month). Only paid invoices, counted by due date.	api.py, assistant.py	storage
app/core/storage.py	SQLite: load a whole table, save (replace) a whole table, next number, find by number.	entities, reports, assistant, api	data/invoices.db
Ollama (separate program)	Runs the models. The four inv-* bots are llama3.2:3b with rules built in from modelfiles/ (temperature 0).	bot.chat, assistant.chat, guide.embed	text, tool calls or vectors

1.4 The stored data

Four tables in data/invoices.db. Numbers are given by Python, never by a model or the user: one above the highest number that exists (storage.next_id).

Table	Columns	Example
customers	id, name, address	CUS-001, Felix Hartmann, Ringstr. 2, 44135 Dortmund
products	id, name, sell_price	PRD-003, Desk Lamp, 29.9
invoices	id, customer_id, date, due_date, status	INV-004, CUS-001, 05/09/2026, 05/10/2026, unpaid
invoice_lines	invoice_id, line, product_id, quantity, unit_price	INV-004, 1, PRD-003, 2, 29.9

In Python an invoice travels as one dictionary with its lines inside: storage.load("invoices") joins invoice_lines into a **lines** list, and storage.save writes them back. The **total** is never stored; invoices.invoice_total adds quantity × unit_price whenever it is needed. The unit_price is copied from the product when the invoice is made, so a later price change does not alter old invoices.

1.5 The objects that travel

These shapes appear again and again. Knowing them makes every scenario easier to follow.

Browser → POST /chat (request model ChatIn; conversation_id is null on the first message)

```
{"message": "add customer Lena Berg, Ringstr. 2, 44135 Dortmund", "conversation_id": null}
```

/chat → Browser, variant 1: a reply

```
{"conversation_id": "7f3a…", "entity": "customer",
 "reply": "What is the address of Lena Berg?",       ← a question, an error or an answer …
 "open": true,                                         ← the task is not finished; the next message goes to the same bot
 "ui": ["new_customer"]}                               ← which form(s) the browser should show
```

/chat → Browser, variant 2: a proposal (shown with Yes / No; on Yes the browser sends exactly method + path + body)

```
{"conversation_id": "7f3a…", "entity": "customer",
 "proposal": {"method": "POST", "path": "/customers",
              "body": {"name": "Lena Berg", "address": "Ringstr. 2, 44135 Dortmund"},
              "summary": "Create customer: Lena Berg | Ringstr. 2, 44135 Dortmund"}}
```

Change bot → bot.py: a command line inside the model's reply text (found by COMMAND_RE)

```
CREATE_CUSTOMER {"name": "Lena Berg", "address": "Ringstr. 2, 44135 Dortmund"}
UPDATE_PRODUCT {"name": "Desk Lamp", "field": "sell_price", "value": 30}
DELETE_INVOICE {"id": "INV-012"}
```

Python ↔ Ollama: POST http://localhost:11434/api/chat

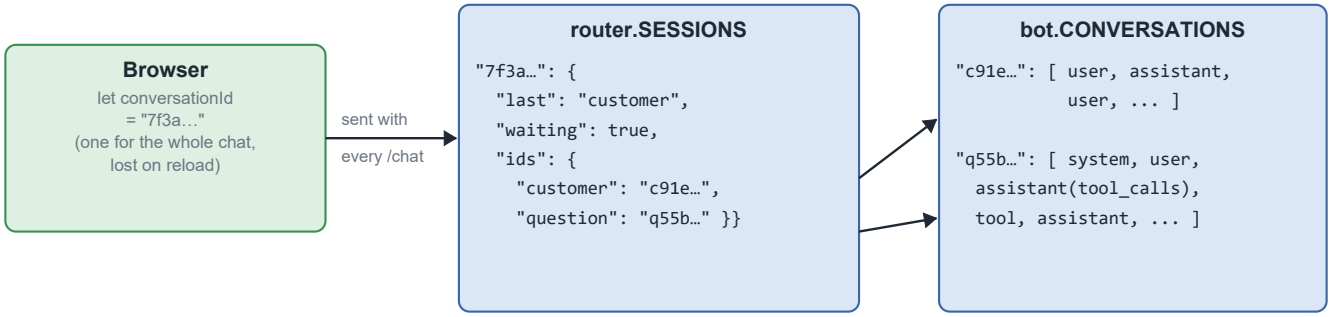
```
request : {"model": "inv-customer-bot", "messages": [{"role": "user", "content": "…"}, …],
           "stream": false, "options": {"temperature": 0}}          (+ "tools": [...] for the assistant)
response: {"message": {"role": "assistant", "content": "…",
                       "tool_calls": [{"function": {"name": "get_customers", "arguments": {}}}]}}
```

2 The engines every scenario reuses

Four functions do the work in almost every scenario: **router.converse**, **bot.converse**, **assistant.converse** and the WRITE endpoints. This part explains each once, so the scenarios can refer to them.

2.1 Conversation memory

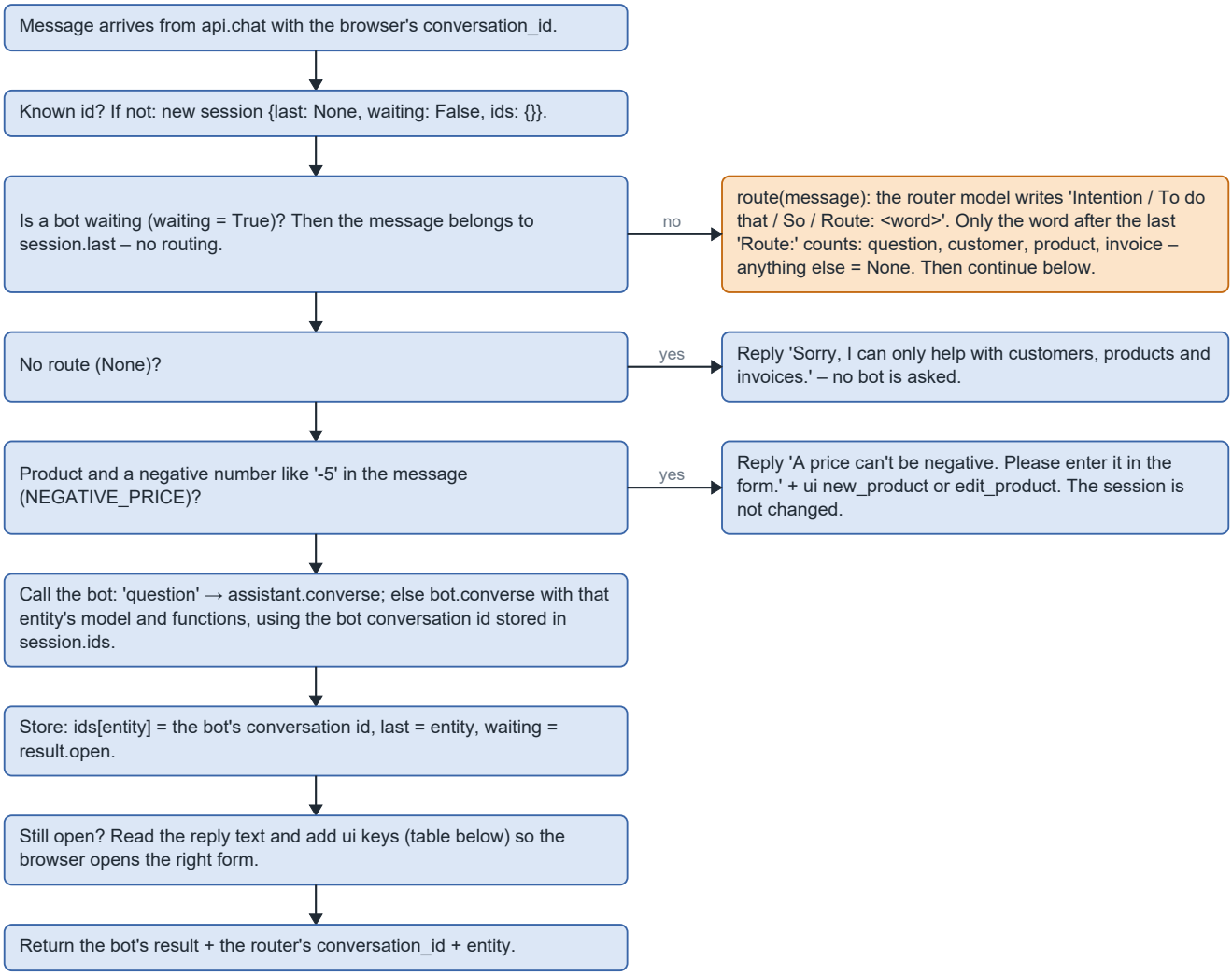
A chat has three levels of memory. All of it lives in the server's RAM, not in the database.



Where	What	Lives until
Browser: **conversationId**	One id for the whole chat. Sent with every /chat message; the server's answer brings it back.	The page is reloaded.
router.py: **SESSIONS**	Per chat: **last** = which bot answered last, **waiting** = is that bot still waiting for an answer, **ids** = one conversation id per bot.	The server restarts.
bot.py: **CONVERSATIONS**	Per bot conversation: the full list of messages. The change bots and the assistant send this whole list to Ollama every time, so the model “remembers”. The list only grows.	The server restarts.

> After a server restart the browser still holds its old conversationId. The router does not know it, quietly starts a new session, and the bots have forgotten everything said before. The router model itself has no memory at all: it only ever sees the one new message.

2.2 router.converse – which bot gets the message



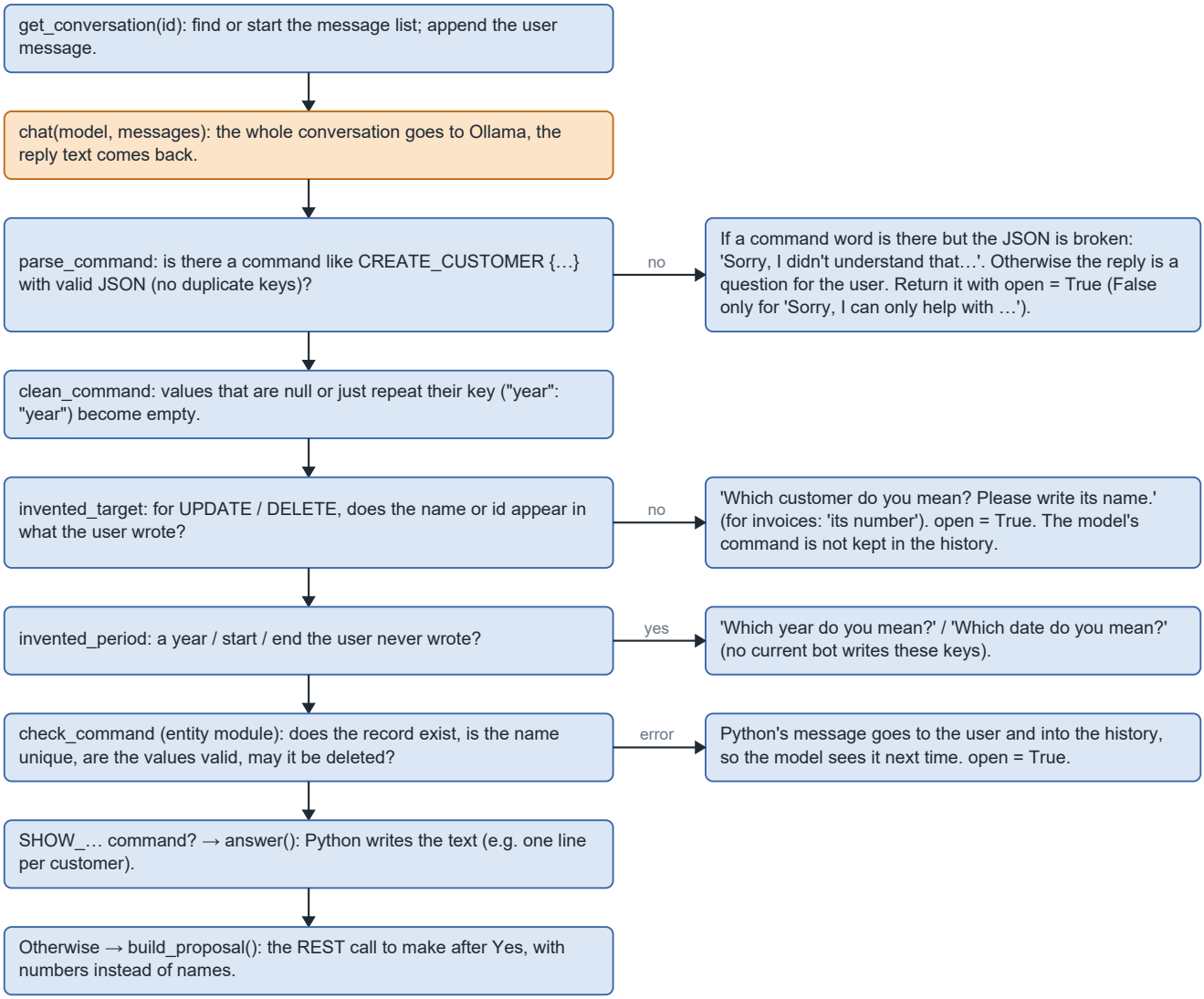
Which reply opens which form (router.converse, only when the bot's result is open)

Bot	If the reply text contains …	ui key	Form in app.js
customer / product	“which customer”, “which product”, “which one do you mean”	customer / product	showRecordPicker – search field of existing records
product	“which product do you want to change” (replaces the key above)	edit_product	showProductEditForm – product + new price
customer	“name of the new customer”, “what is the address of”	new_customer	showCustomerForm – name + address
product	“name of the new product”, “what is the sell price of”	new_product	showProductForm – name + sell price
invoice	“invoice number” / “which invoice”	invoice	showInvoicePicker
invoice	“paid or unpaid”	status	showStatusPicker – two buttons
invoice	“customer”	customer	showRecordPicker("customer")
invoice	“date”	dates	showDatePicker – invoice date + due date
invoice	“product”, “line”, “quantity”, “item”	lines	showLinePicker
assistant	(not by text: the tool the model called, see 2.4)	turnover, product_turnover, monthly_turnover	showTurnoverPicker / showMonthlyTurnoverPicker

For the invoice bot several keys can match at once; the browser then shows several forms.

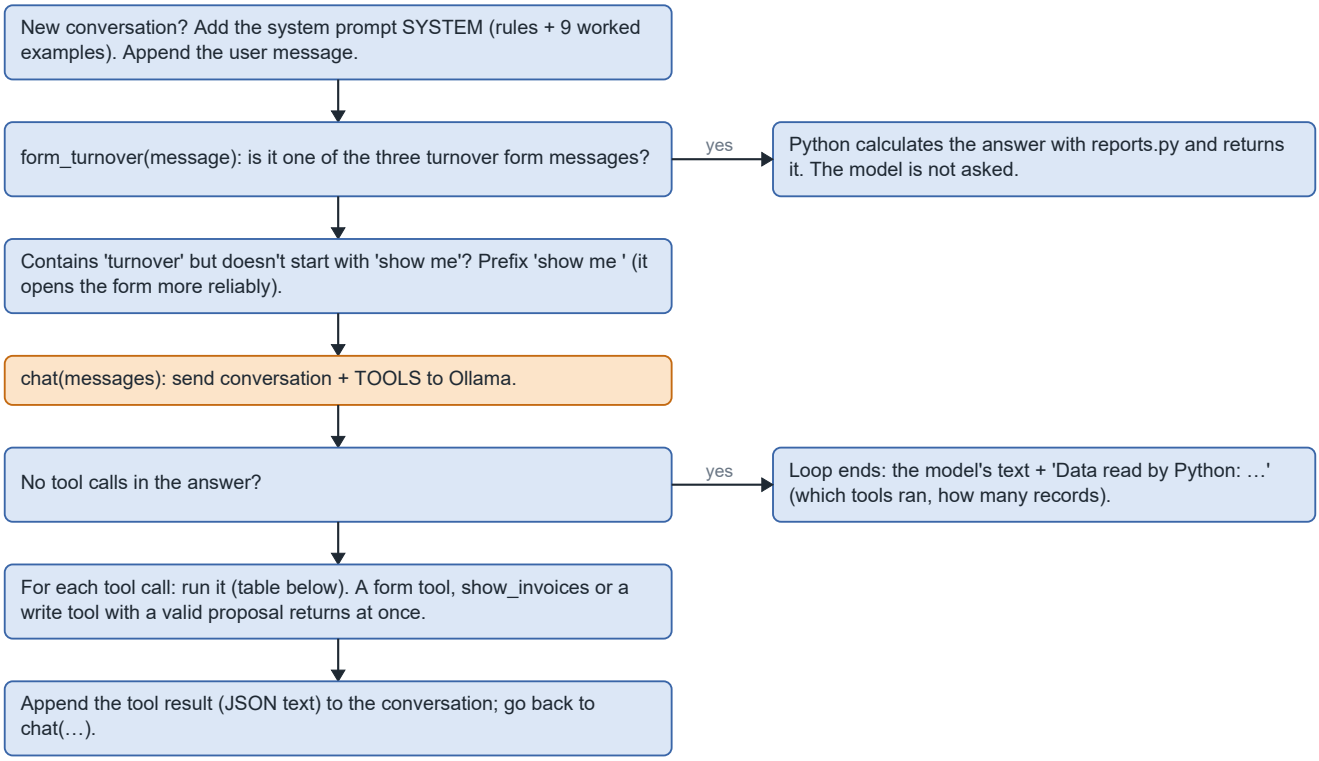
2.3 bot.converse – the change bots and their guards

Used for the customer, product and invoice bots. The same code serves all three: the router passes in the model name and three functions of the entity module – **check_command**, **build_proposal** and **answer**. The model's reply passes a chain of guards; each guard can stop it and send Python's own text to the user instead.



2.4 assistant.converse – the question bot with tools

Used for every message the router calls a **question**. Here the model does not write a command line: Ollama gets a list of **tools** (functions with parameters) and the model answers with *tool calls*. Python runs each tool and puts the result into the conversation as a message with role “tool”. Then the model is asked again – up to MAX_ROUNDS = 5 times per user message.



What Python does with each tool

Tool the model calls	What Python does	Back to
ask_turnover_period, ask_product_turnover_period, ask_monthly_turnover	Return the reply text + ui turnover / product_turnover / monthly_turnover, open = True.	Browser (form)
get_turnover_by_customer, …_by_product, …_by_month	Replaced by the matching ask_… tool (FORM_FOR): a typed turnover question always gets the form.	Browser (form)
show_invoices(status?, invoice_id?)	list_invoices(): Python writes one line per invoice (+ “N unpaid invoices.”) and returns it as the reply. The model never copies the list.	Browser
get_customers, get_products	storage.load(…). If the model invented a filter: all records + a note “has no filter”.	Model
get_invoices(status?)	{count, records}: Python counts, every invoice with its total.	Model
create_ / update_ / delete_ customer, product, invoice	propose(): record must exist (update/delete), values checked with check_customer / check_product / check_invoice → a proposal, or “Not possible: …” back to the model.	Browser (proposal) or model (error)

2.5 Saving – the WRITE endpoints

After Yes the browser calls one of these. They are ordinary REST endpoints: any program could call them, and they trust nothing that came before.

Step	Done by	On failure
1. Body has the right fields and types (CustomerIn, ProductIn, InvoiceIn)	FastAPI / pydantic	422
2. Values valid again: check_customer, check_product, check_invoice	api.py → entity module	400 + the message
3. Record exists (PUT, DELETE); for DELETE also: not used on an invoice / not paid	entity module (update_… / delete_…)	404 / 400
4. storage.load the whole table, change the list in Python	entity module	–
5. storage.save: DELETE all rows of the table, INSERT every record again (invoices: also all invoice_lines)	storage.py	–
6. The new or changed record as JSON (invoices with total)	api.py → browser	–

The browser shows “Saved as CUS-013.”, “Changed PRD-003.” or “Deleted INV-012.” and reloads the table (loadList → GET /customers etc.). An error is shown as “Not saved: …”.

2.6 The form contract

Every form ends by sending one chat message. The wording is fixed; this table is the contract between app.js and the bots / Python.

ui key → form	Data the form loads	Message it sends (example)	Read by
customer / product → showRecordPicker	GET /customers or /products	`customer: Karl Becker`	the waiting change bot
new_customer → showCustomerForm	– (name taken from the bot's question)	`add customer Lena Berg, Ringstr. 2, 44135 Dortmund`	customer bot
new_product → showProductForm	–	`new product Tea Cup, sell price 4.50`	product bot
edit_product → showProductEditForm	GET /products	`product: Desk Lamp, new sell price 30.00`	product bot
dates → showDatePicker	– (invoice date ≥ today, due date ≥ invoice date)	`invoice date 21/10/2026, due date 20/11/2026`	invoice bot
lines → showLinePicker	GET /products	`products: 2 Desk Lamp, 1 USB Cable 1m`	invoice bot
invoice → showInvoicePicker	GET /invoices, GET /customers	`INV-004`	invoice bot
status → showStatusPicker	–	`paid` or `unpaid`	invoice bot
turnover → showTurnoverPicker("customer")	GET /customers, GET /products	`turnover per customer from 01/01/2026 to 31/10/2026, customer: Felix Hartmann (CUS-001)`	assistant.form_turnover (Python)
product_turnover → showTurnoverPicker("product")	GET /products, GET /customers	`turnover per product from 01/01/2026 to 31/10/2026, all products, customer: Karl Becker (CUS-002)`	assistant.form_turnover
monthly_turnover → showMonthlyTurnoverPicker	GET /customers, GET /products	`monthly turnover for 2026, all customers, product: Desk Lamp (PRD-003)`	assistant.form_turnover

The turnover forms send “name (id)”, so Python can use the id and two records with the same name stay apart. The other choosers send only the name; the entity module looks it up (exact name first).

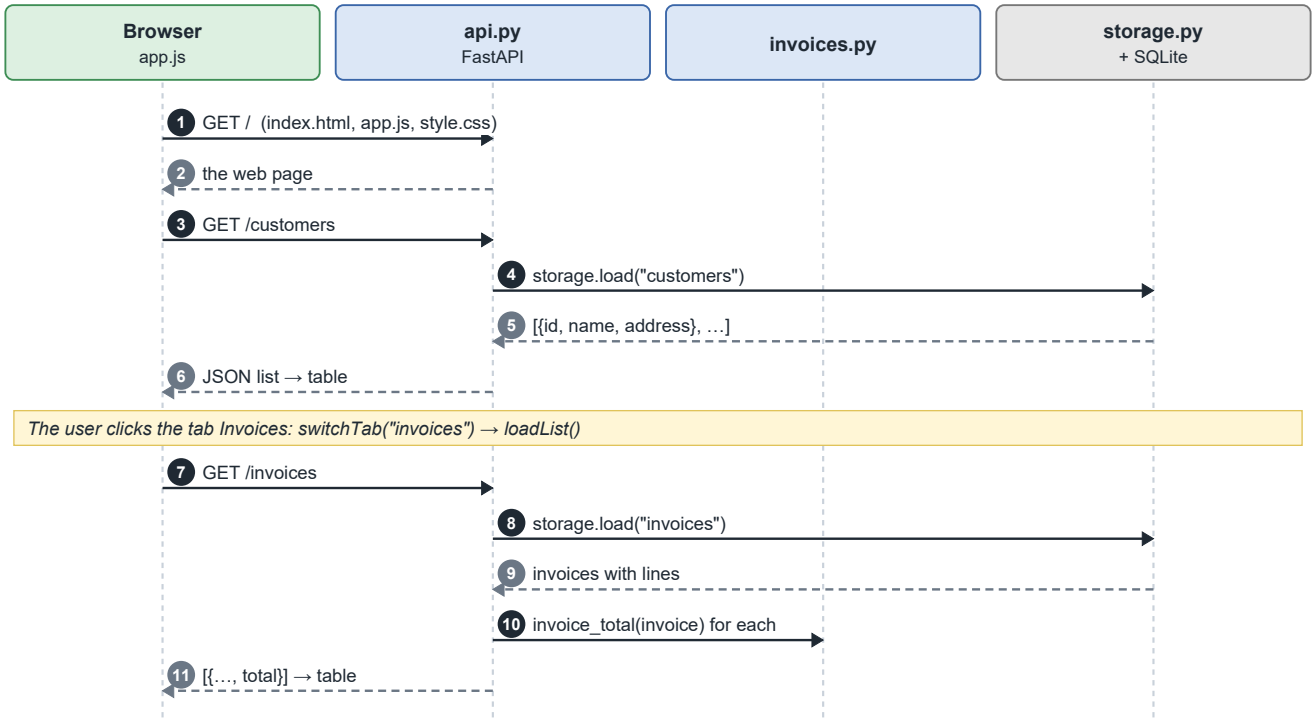
3 Scenarios

Each scenario starts with what the user does, gives the path in one sentence, then shows the sequence and explains every arrow. Later scenarios shorten what earlier ones already showed.

#	Scenario	Models used	Saves?
S1	Open the page and look at the tables	none	no
S2	Create a customer in one message	router, customer bot	after Yes
S3	Create a customer, address missing → form	router, customer bot	after Yes
S4	Change a product's price with the edit form	router, product bot	after Yes
S5	A negative price is stopped before any bot	router	no
S6	Delete a customer who still has invoices	router, customer bot	no (refused)
S7	A name matches two customers → chooser	router, customer bot	after Yes
S8	The bot invents the record → Python asks	router, customer bot	after Yes
S9	Create an invoice step by step (4 forms)	router, invoice bot	after Yes
S10	Mark an invoice as paid	router, invoice bot	after Yes
S11	Ask a question: where does a customer live?	router, assistant	no
S12	List unpaid invoices / is one invoice paid?	router, assistant	no
S13	Turnover per customer (form, Python answers)	router, assistant (1st turn)	no
S14	Monthly turnover and the product variant	router, assistant (1st turn)	no
S15	A change through the assistant (write tool)	router, assistant	after Yes
S16	Yes, but the data changed in the meantime	–	refused
S17	The Help button	embedding model, llama3.2:3b	no
S18	A message that is not about invoicing	router	no
S19	Other entry points: direct bot chats, reports	depends	–

S1 Open the page and look at the tables

The user	Opens http://127.0.0.1:8000 and clicks the tabs Customers, Products, Invoices.
The path	Plain REST: the browser reads lists through GET endpoints; api.py reads them from storage. No model, no chat.



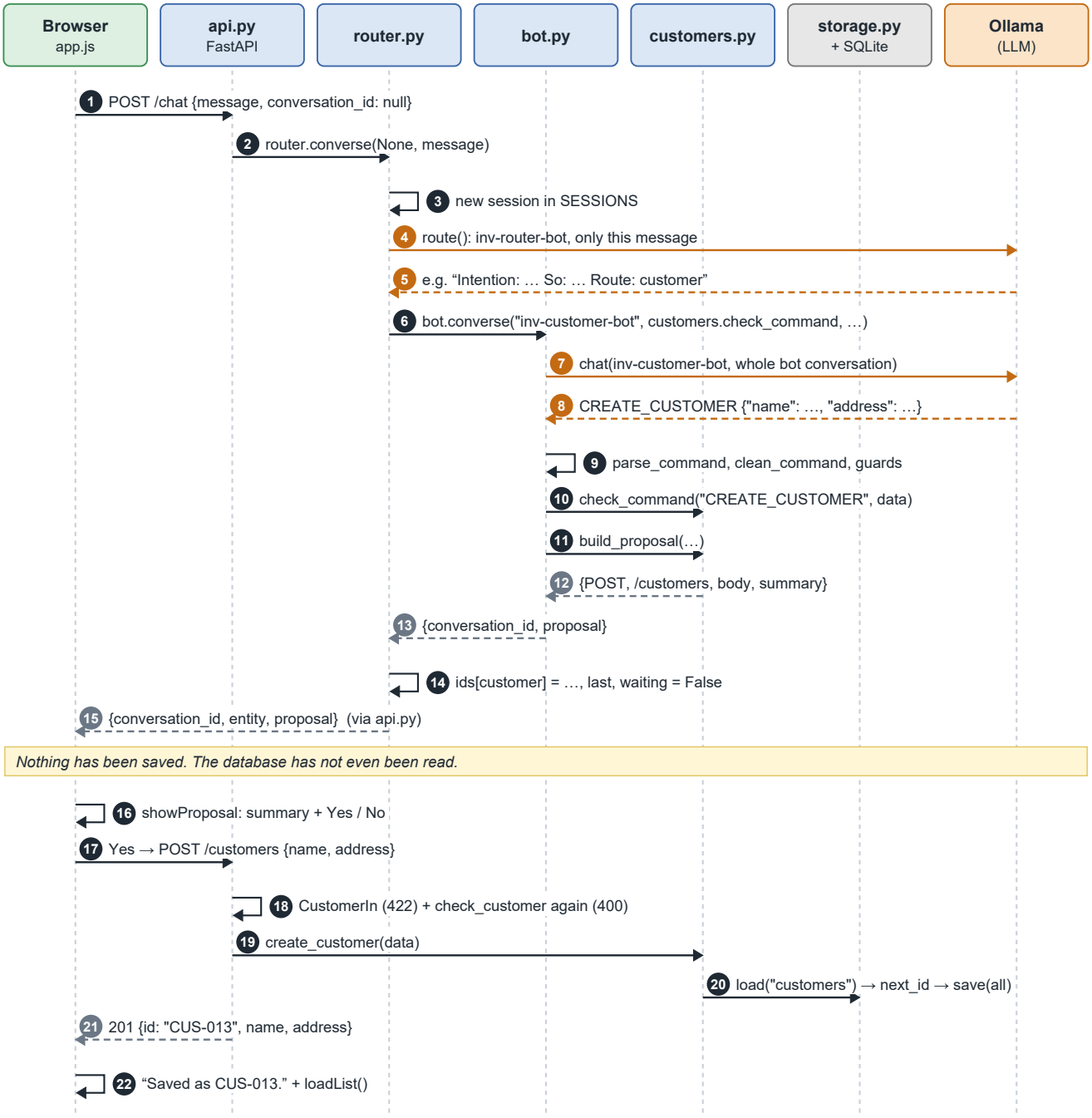
Step by step

#	From → to	What happens
1	Browser → api.py	**GET / (index.html, app.js, style.css)** The last line of api.py mounts static/ at “/”, after all API routes, so /customers etc. are matched first.
2	api.py → Browser	**the web page** app.js starts: entity = "customers", conversationId = null, then loadList().
3	Browser → api.py	**GET /customers** loadList() asks for the active tab.
4	api.py → storage.py	**storage.load("customers")** Opens data/invoices.db (created with its 4 tables if missing), SELECT * ORDER BY id.
5	storage.py → api.py	**[{id, name, address}, …]**
6	api.py → Browser	**JSON list → table** COLUMNS in app.js decides which fields become columns.
		The user clicks the tab Invoices: switchTab("invoices") → loadList()
7	Browser → api.py	**GET /invoices**
8	api.py → storage.py	**storage.load("invoices")** Loads the invoices and, for each, its rows from invoice_lines as a list “lines”.
9	storage.py → api.py	**invoices with lines**
10	api.py → invoices.py	**invoice_total(invoice) for each** api.list_invoices adds “total” = Σ quantity × unit_price, rounded to the cent.
11	api.py → Browser	**[{…, total}] → table**

> The tables never refresh by themselves. They are reloaded after every successful Yes (confirmProposal → loadList) and when a tab is clicked.

S2 Create a customer in one message

The user	Types “add customer Lena Berg, Ringstr. 2, 44135 Dortmund” and presses Send, then clicks Yes.
The path	router model → “customer” · customer bot → CREATE_CUSTOMER line · bot.py guards + customers.py check → proposal · browser shows Yes/No · Yes → POST /customers → saved.



Step by step

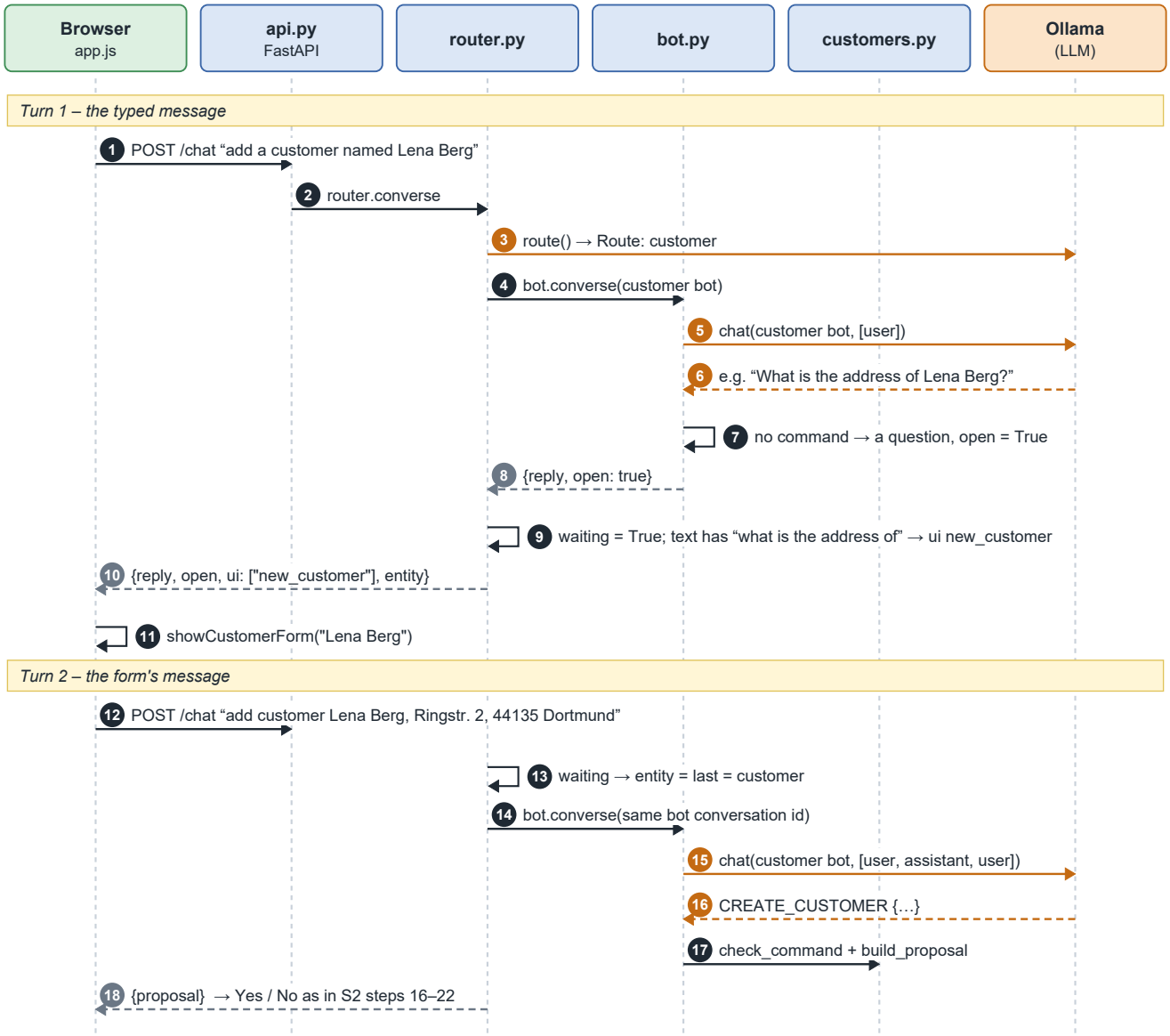
#	From → to	What happens
1	Browser → api.py	**POST /chat {message, conversation_id: null}** sendMessage() shows “You: …” and a waiting “Bot: …”, then posts the text.
2	api.py → router.py	**router.converse(None, message)** api.chat only forwards. FastAPI has already checked the body against ChatIn.
3	router.py	**new session in SESSIONS** Unknown id → new uuid, {last: None, waiting: False, ids: {}}. Nobody is waiting, so the message must be routed.

#	From → to	What happens
4	router.py → Ollama	**route(): inv-router-bot, only this message** Through bot.chat. The router model gets no history.
5	Ollama → router.py	**e.g. “Intention: … So: … Route: customer”** route() lower-cases the reply, takes the words after the last “route:” and keeps the first known one → "customer".
6	router.py → bot.py	**bot.converse("inv-customer-bot", customers.check_command, …)** BOTS maps "customer" to the model name and the module customers.py; its three functions are passed in. session.ids has no customer conversation yet → None.
7	bot.py → Ollama	**chat(inv-customer-bot, whole bot conversation)** New entry in CONVERSATIONS; the user message is appended. The bot's rules and examples live in modelfiles/Modelfile-customer, built into the model.
8	Ollama → bot.py	**CREATE_CUSTOMER {"name": …, "address": …}**
9	bot.py	**parse_command, clean_command, guards** The command and its JSON are found. invented_target does not apply to CREATE. invented_period finds no year / start / end.
10	bot.py → customers.py	**check_command("CREATE_CUSTOMER", data)** → check_customer: name and address are there and not “...”. Returns None = valid.
11	bot.py → customers.py	**build_proposal(…)**
12	customers.py → bot.py	**{POST, /customers, body, summary}**
13	bot.py → router.py	**{conversation_id, proposal}** The bot's reply is added to its history. No “open” key: the task is finished.
14	router.py	**ids[customer] = …, last, waiting = False** No ui keys: they are only added to open replies.
15	router.py → Browser	**{conversation_id, entity, proposal} (via api.py)**
		Nothing has been saved. The database has not even been read.
16	Browser	**showProposal: summary + Yes / No** “Bot (customer): Create customer: Lena Berg | Ringstr. 2, 44135 Dortmund [Yes] [No]”. No → “Cancelled. Nothing was saved.”, no request at all.
17	Browser → api.py	**Yes → POST /customers {name, address}** confirmProposal() sends proposal.method, path and body unchanged.
18	api.py	**CustomerIn (422) + check_customer again (400)**
19	api.py → customers.py	**create_customer(data)**
20	customers.py → storage.py	**load("customers") → next_id → save(all)** next_id: highest number + 1, e.g. CUS-013. save deletes all rows and inserts them all again.
21	api.py → Browser	**201 {id: "CUS-013", name, address}**
22	Browser	**“Saved as CUS-013.” + loadList()**

> Two independent checks: check_customer runs once when the proposal is built and again in the WRITE endpoint. The second one matters when the body was changed on the way or the endpoint is called by another program.

S3 Create a customer, the address is missing

The user	Types “add a customer named Lena Berg”. A form with name and address appears; the user fills in the address and clicks “Create customer”, then Yes.
The path	Turn 1: the customer bot asks, the router sees the question and adds ui new_customer · turn 2: the form sends a full sentence, the router skips routing because the customer bot is waiting · then as S2.



Step by step

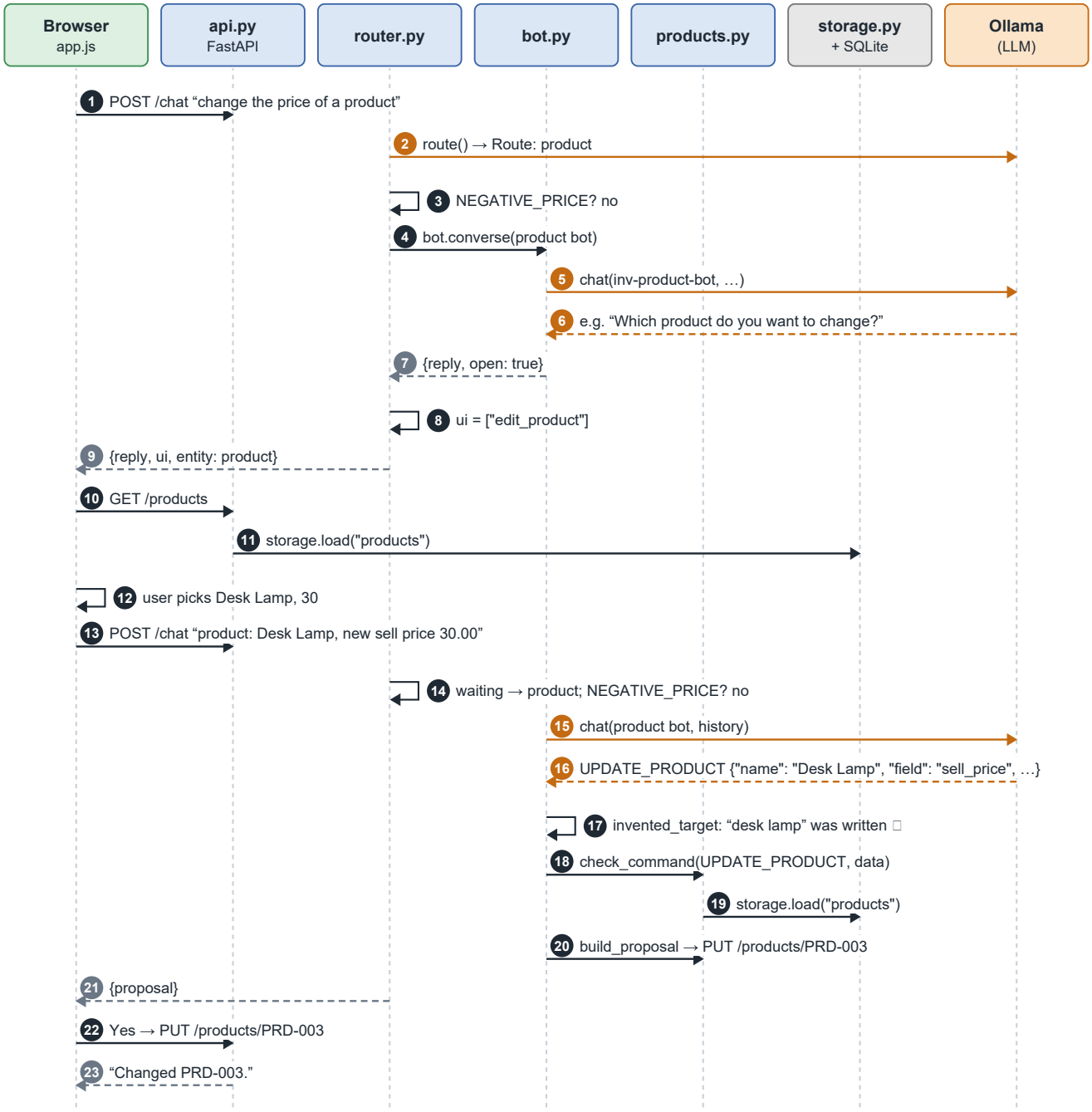
#	From → to	What happens
		Turn 1 – the typed message
1	Browser → api.py	**POST /chat “add a customer named Lena Berg”**
2	api.py → router.py	**router.converse**
3	router.py → Ollama	**route() → Route: customer**
4	router.py → bot.py	**bot.converse(customer bot)**
5	bot.py → Ollama	**chat(customer bot, [user])**
6	Ollama → bot.py	**e.g. “What is the address of Lena Berg?”**
7	bot.py	**no command → a question, open = True** parse_command finds nothing and there is no command word, so the reply is a question. It is added to the history.

#	From → to	What happens
8	bot.py → router.py	**{reply, open: true}**
9	router.py	**waiting = True; text has “what is the address of” → ui new_customer**
10	router.py → Browser	**{reply, open, ui: ["new_customer"], entity}**
11	Browser	**showCustomerForm("Lena Berg")** app.js takes the name out of the question with the pattern “address of (.+?)\?” and fills it in. The button is enabled only when both fields have text.
		Turn 2 – the form's message
12	Browser → api.py	**POST /chat “add customer Lena Berg, Ringstr. 2, 44135 Dortmund”** The same conversation_id as turn 1.
13	router.py	**waiting → entity = last = customer** route() is not called: the answer belongs to the bot that asked.
14	router.py → bot.py	**bot.converse(same bot conversation id)**
15	bot.py → Ollama	**chat(customer bot, [user, assistant, user])**
16	Ollama → bot.py	**CREATE_CUSTOMER {…}**
17	bot.py → customers.py	**check_command + build_proposal**
18	router.py → Browser	**{proposal} → Yes / No as in S2 steps 16–22**

> **waiting** is the key to multi-turn tasks: as long as a bot's last reply was open (a question or an error), every next message goes straight to that bot – even if the user changes the subject.

S4 Change a product's price with the edit form

The user	Types “change the price of a product”, picks Desk Lamp in the form, enters 30, clicks “Change price”, then Yes.
The path	product bot asks which product · router opens edit_product · form sends “product: Desk Lamp, new sell price 30.00” · UPDATE_PRODUCT → checks → PUT /products/PRD-003 after Yes.



Step by step

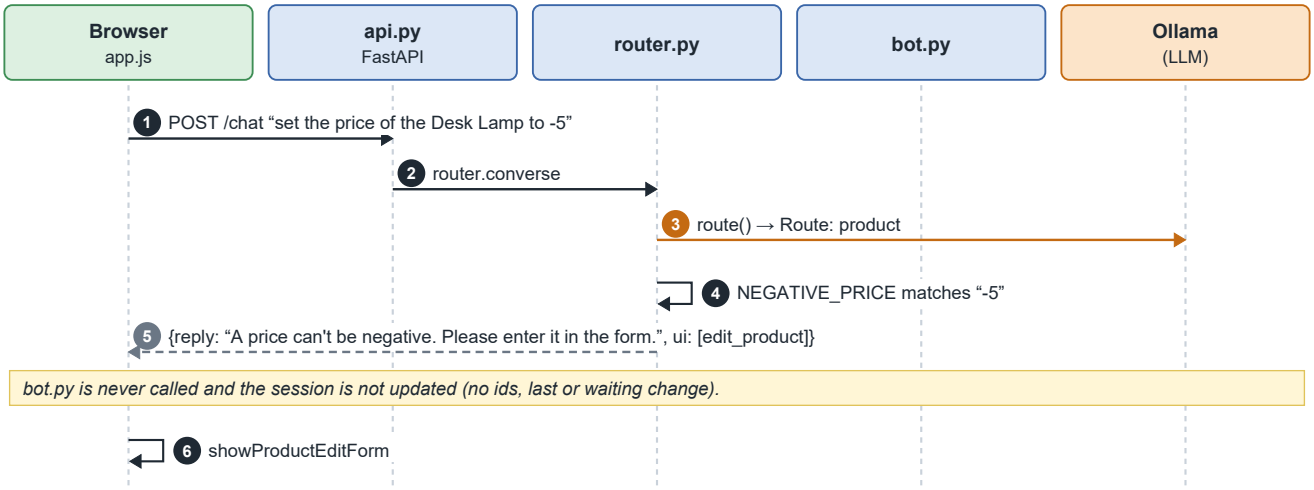
#	From → to	What happens
1	Browser → api.py	**POST /chat “change the price of a product”**
2	router.py → Ollama	**route() → Route: product**
3	router.py	**NEGATIVE_PRICE? no**
4	router.py → bot.py	**bot.converse(product bot)**
5	bot.py → Ollama	**chat(inv-product-bot, …)**
6	Ollama → bot.py	**e.g. “Which product do you want to change?”**

#	From → to	What happens
7	bot.py → router.py	**{reply, open: true}**
8	router.py	**ui = ["edit_product"]** “which product” would first give the plain chooser; the later rule for “which product do you want to change” replaces it with the edit form.
9	router.py → Browser	**{reply, ui, entity: product}**
10	Browser → api.py	**GET /products** showProductEditForm fills its search list.
11	api.py → storage.py	**storage.load("products")**
12	Browser	**user picks Desk Lamp, 30** The button works only for an existing product name and a price ≥ 0.
13	Browser → api.py	**POST /chat “product: Desk Lamp, new sell price 30.00”**
14	router.py	**waiting → product; NEGATIVE_PRICE? no**
15	bot.py → Ollama	**chat(product bot, history)**
16	Ollama → bot.py	**UPDATE_PRODUCT {"name": "Desk Lamp", "field": "sell_price", …}**
17	bot.py	**invented_target: “desk lamp” was written** □
18	bot.py → products.py	**check_command(UPDATE_PRODUCT, data)** find_products("Desk Lamp"): exact name first → one product. field is name or sell_price; check_product on the product with the new value (number, ≥ 0).
19	products.py → storage.py	**storage.load("products")**
20	bot.py → products.py	**build_proposal → PUT /products/PRD-003** body = the full product with the one field changed: {name: "Desk Lamp", sell_price: 30.0}. summary “Change sell_price of PRD-003 (Desk Lamp): '29.9' -> '30.0'”.
21	router.py → Browser	**{proposal}**
22	Browser → api.py	**Yes → PUT /products/PRD-003** api.change_product: check_product → update_product → storage.save("products").
23	api.py → Browser	**“Changed PRD-003.”**

> Invoices are not touched: every invoice line stores its own unit_price. The new price is used only by invoices made from now on.

S5 A negative price is stopped before any bot

The user	Types “set the price of the Desk Lamp to -5”.
The path	router model → product · router.py sees “-5” and answers itself with the edit form · the product bot is never asked.



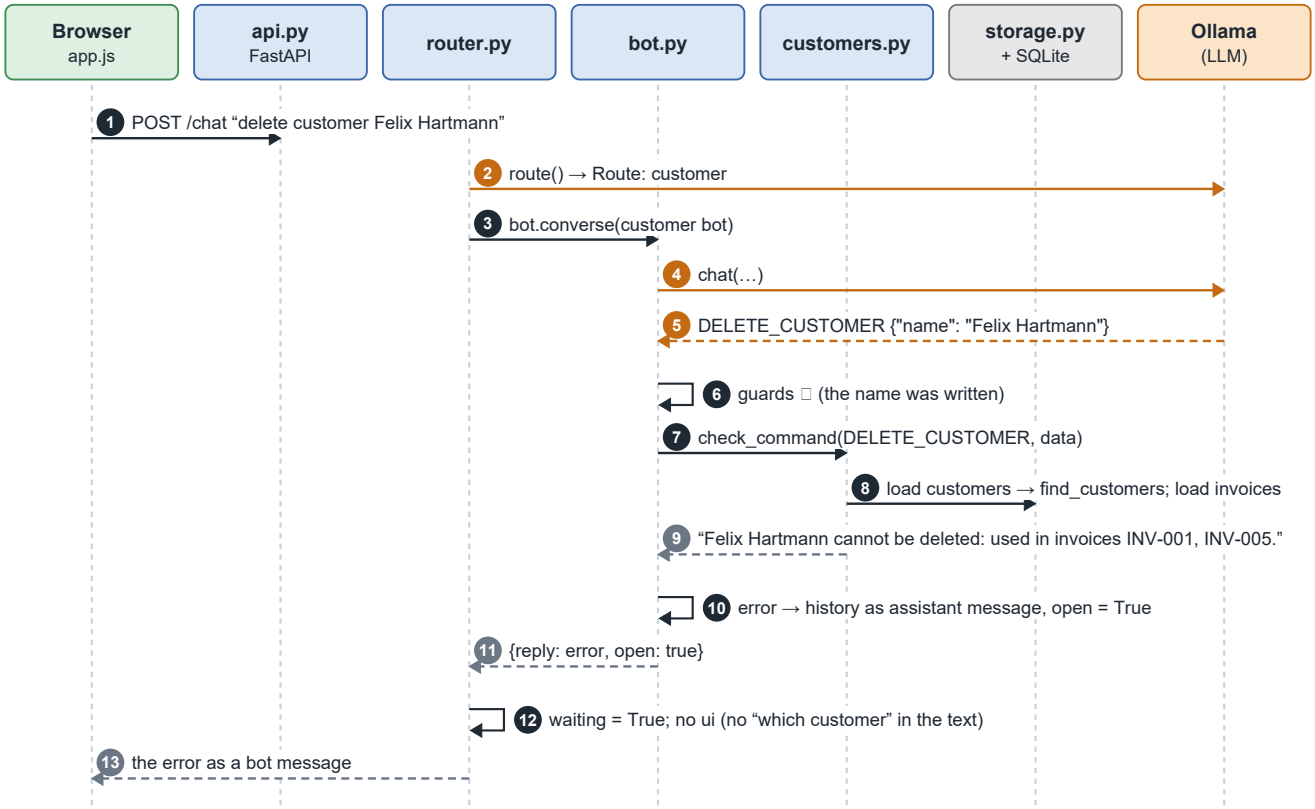
Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “set the price of the Desk Lamp to -5”**
2	api.py → router.py	**router.converse**
3	router.py → Ollama	**route() → Route: product**
4	router.py	**NEGATIVE_PRICE matches “-5”** The pattern (?<!\w)-\s*\d finds a minus before a digit, but not a dash inside a word such as “USB-3 Cable”. The message has no “create”, “add” or “new” → the edit form.
5	router.py → Browser	**{reply: “A price can't be negative. Please enter it in the form.”, ui: [edit_product]}**
		bot.py is never called and the session is not updated (no ids, last or waiting change).
6	Browser	**showProductEditForm** The form's price field has min = 0 and the button stays disabled for a negative price.

Why here and not in the bot? A small model given “-5” tends to refuse or to drop the sign. Catching it in Python before the model sees it makes the answer the same every time. The last line of defence is still check_product (“The sell_price must not be negative.”) in the proposal and in PUT/POST /products.

S6 Delete a customer who still has invoices

The user	Types “delete customer Felix Hartmann”.
The path	customer bot → DELETE_CUSTOMER · customers.check_command finds invoices of this customer · Python's message goes to the user; no proposal, no Yes button.



Step by step

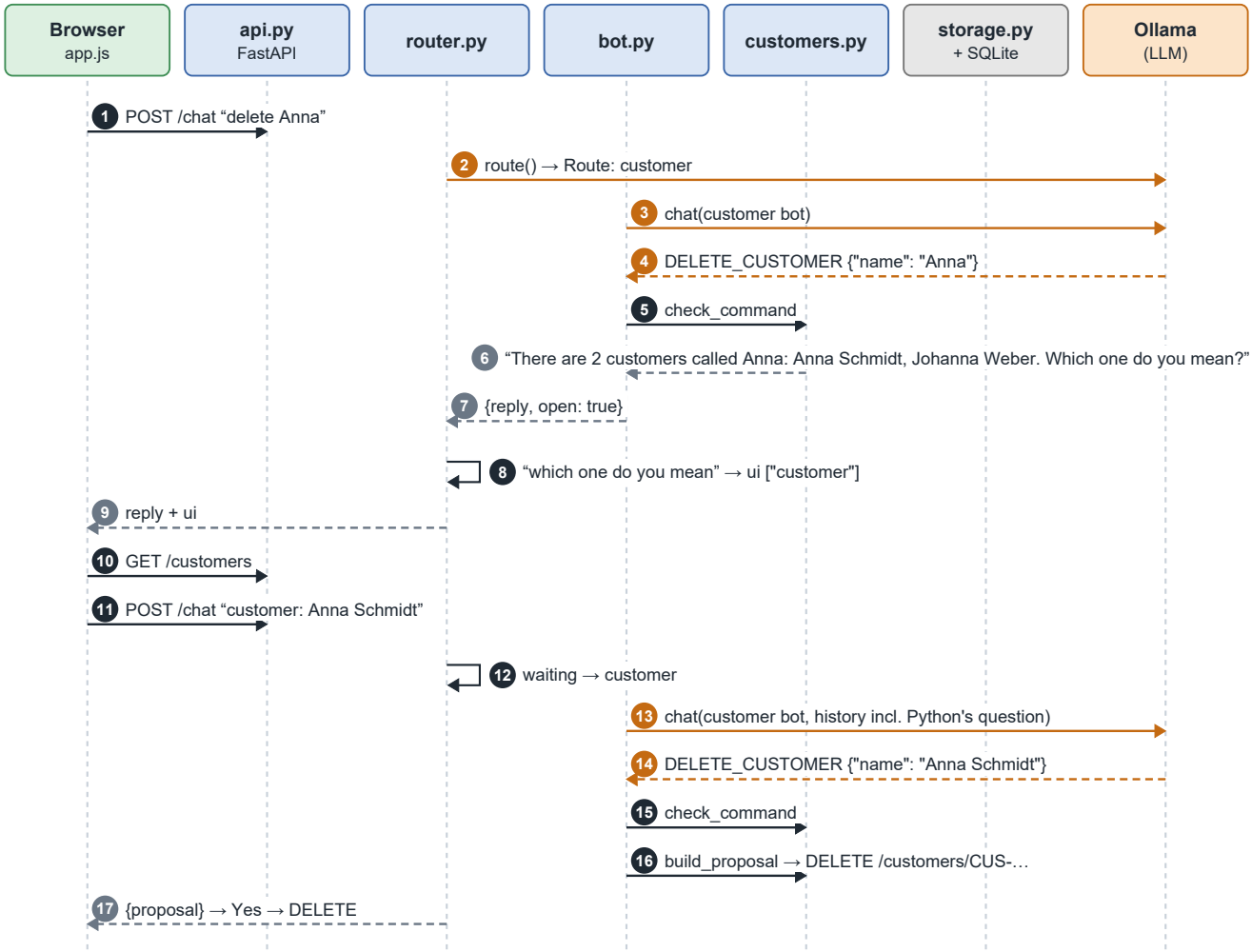
#	From → to	What happens
1	Browser → api.py	**POST /chat “delete customer Felix Hartmann”**
2	router.py → Ollama	**route() → Route: customer**
3	router.py → bot.py	**bot.converse(customer bot)**
4	bot.py → Ollama	**chat(…)**
5	Ollama → bot.py	**DELETE_CUSTOMER {"name": "Felix Hartmann"}**
6	bot.py	**guards □ (the name was written)**
7	bot.py → customers.py	**check_command(DELETE_CUSTOMER, data)**
8	customers.py → storage.py	**load customers → find_customers; load invoices** One customer matches. Then every invoice with this customer_id is collected.
9	customers.py → bot.py	**“Felix Hartmann cannot be deleted: used in invoices INV-001, INV-005.”**
10	bot.py	**error → history as assistant message, open = True** The model sees Python's sentence as if it had said it, so a later turn can build on it.
11	bot.py → router.py	**{reply: error, open: true}**
12	router.py	**waiting = True; no ui (no “which customer” in the text)**
13	router.py → Browser	**the error as a bot message**

The same rule exists three times, on purpose: in check_command (the chat refuses early), in delete_customer (raises ValueError → api.py answers 400) and indirectly in the database's REFERENCES. Products follow the same pattern (“used in invoices …”), and a paid invoice is refused with “Invoice INV-011 cannot be deleted: it is paid.”

> Because the error reply is open, the customer bot stays “waiting”: the next message – whatever it is about – goes to the customer bot without routing.

S7 A name matches two customers → chooser

The user	Types “delete Anna”. Imagine two customers whose names contain “anna”: Anna Schmidt and Johanna Weber.
The path	check_command finds two matches and asks · the router sees “which one do you mean” and adds the customer chooser · the chooser sends the exact name · exact match wins → proposal.



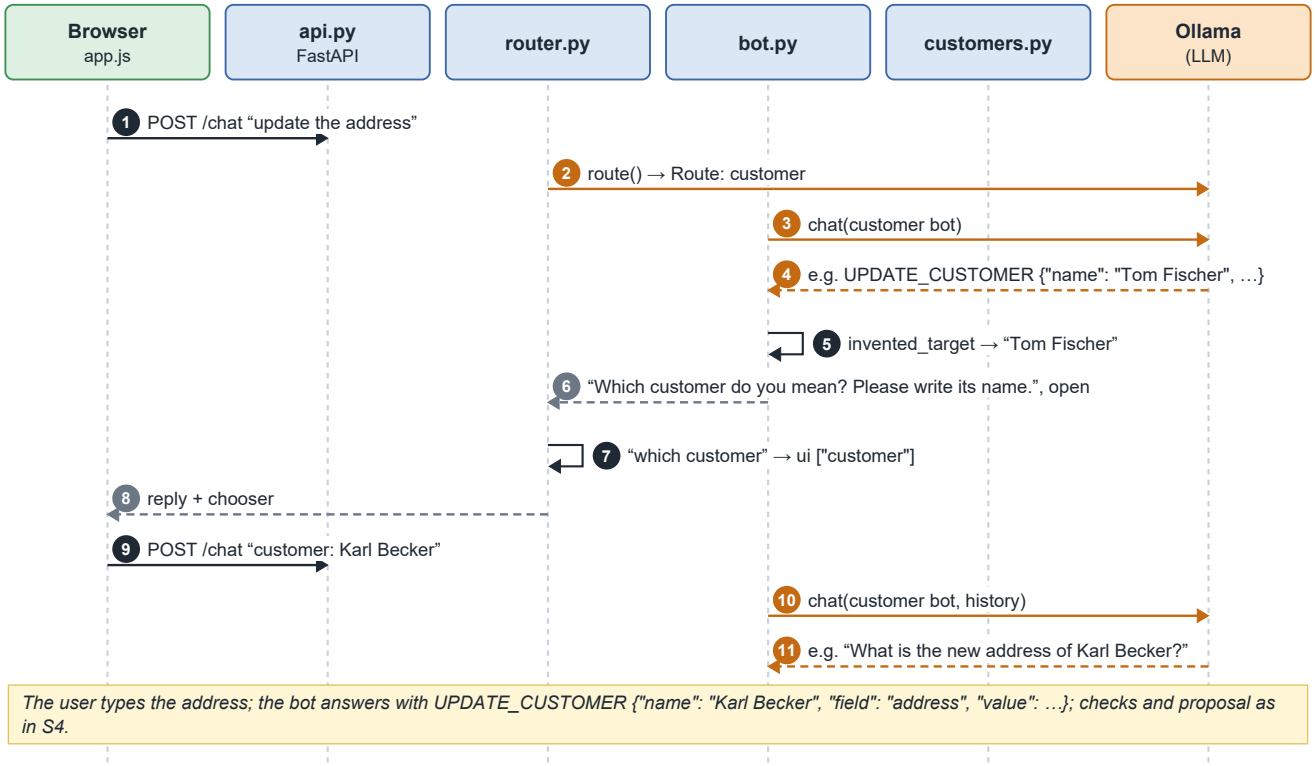
Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “delete Anna”**
2	router.py → Ollama	**route() → Route: customer**
3	bot.py → Ollama	**chat(customer bot)**
4	Ollama → bot.py	**DELETE_CUSTOMER {"name": "Anna"}**
5	bot.py → customers.py	**check_command** find_customers("Anna"): every name that contains “anna”, not case-sensitive → 2 records.
6	customers.py → bot.py	**“There are 2 customers called Anna: Anna Schmidt, Johanna Weber. Which one do you mean?”**
7	bot.py → router.py	**{reply, open: true}**
8	router.py	**“which one do you mean” → ui ["customer"]**
9	router.py → Browser	**reply + ui**
10	Browser → api.py	**GET /customers** showRecordPicker: all customers A–Z, the address as hint. Only an existing name enables the button.
11	Browser → api.py	**POST /chat “customer: Anna Schmidt”**
12	router.py	**waiting → customer**

#	From → to	What happens
13	bot.py → Ollama	**chat(customer bot, history incl. Python's question)**
14	Ollama → bot.py	**DELETE_CUSTOMER {"name": "Anna Schmidt"}**
15	bot.py → customers.py	**check_command** find_customers: “Anna Schmidt” matches one name exactly → only that one. No invoices → None.
16	bot.py → customers.py	**build_proposal → DELETE /customers/CUS-…**
17	router.py → Browser	**{proposal} → Yes → DELETE**

S8 The bot invents the record → Python asks

The user	Types “update the address” – no customer named.
The path	The customer bot sometimes copies a name from its own examples (here Tom Fischer) · invented_target sees that the user never wrote it · Python asks instead · the chooser fills the gap.



Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “update the address”**
2	router.py → Ollama	**route() → Route: customer**
3	bot.py → Ollama	**chat(customer bot)**
4	Ollama → bot.py	**e.g. UPDATE_CUSTOMER {"name": "Tom Fischer", …}** “Tom Fischer” is an example name in Modelfile-customer (a known problem, open_problems #15).
5	bot.py	**invented_target → “Tom Fischer”** It joins everything the user wrote in this bot conversation, lower-cased, and looks for the name / id of the command in it. Not found → the command is dropped.
6	bot.py → router.py	**“Which customer do you mean? Please write its name.”, open** Python's question – not the bot's command – goes into the history.
7	router.py	**“which customer” → ui ["customer"]**
8	router.py → Browser	**reply + chooser**
9	Browser → api.py	**POST /chat “customer: Karl Becker”**
10	bot.py → Ollama	**chat(customer bot, history)**
11	Ollama → bot.py	**e.g. “What is the new address of Karl Becker?”**
		The user types the address; the bot answers with UPDATE_CUSTOMER {"name": "Karl Becker", "field": "address", "value": …}; checks and proposal as in S4.

The other guards in the same place

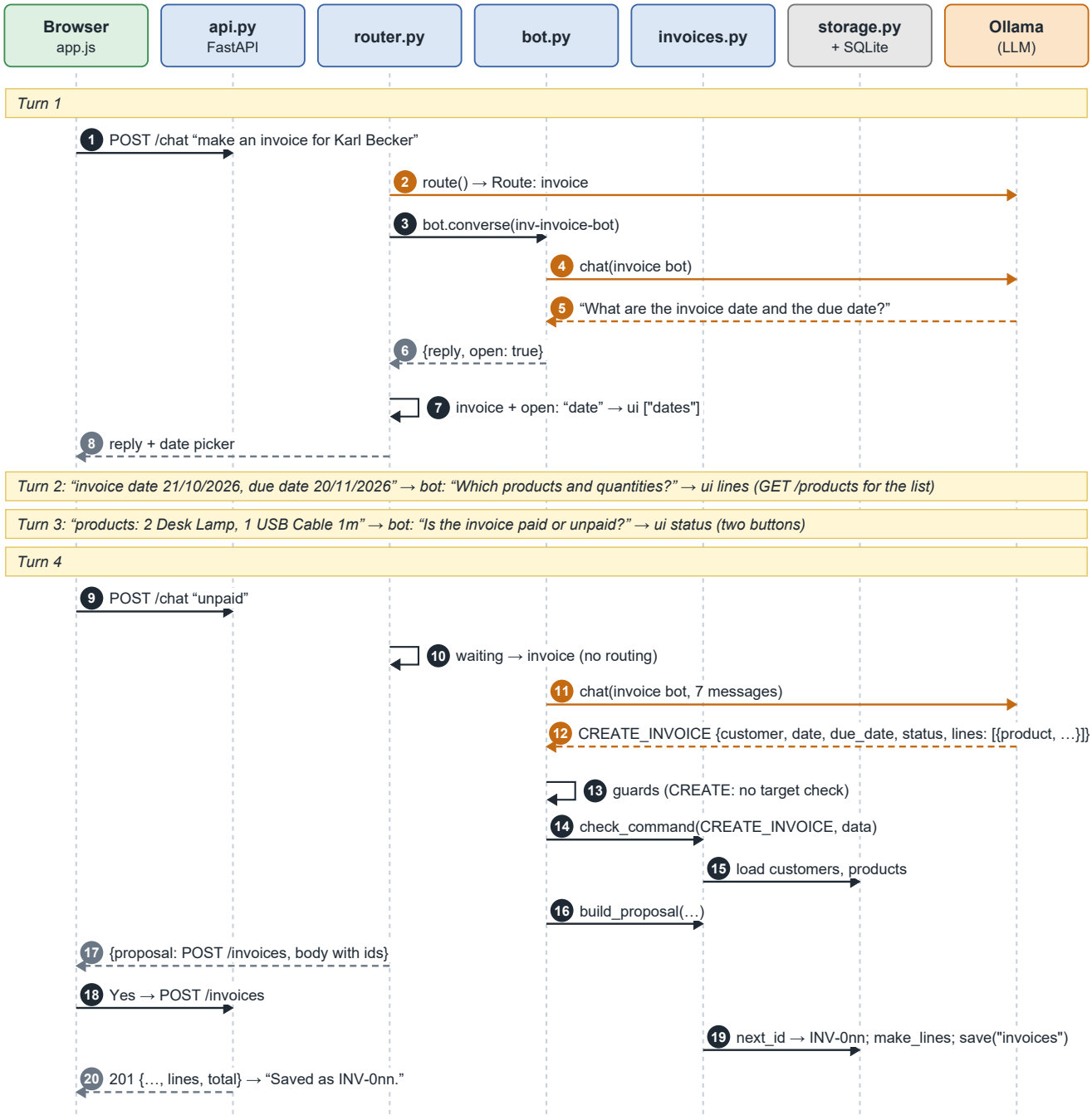
Guard	Catches	Answer
parse_command fails but COMMAND_RE matches	A command word with broken or duplicate-key JSON	“Sorry, I didn't understand that. Please say it again in other words.”

Guard	Catches	Answer
clean_command	"address": null, "year": "year"	the value becomes empty, so check_customer says “The address is missing.”
invented_target	UPDATE / DELETE on a name or id the user never wrote	“Which customer / product do you mean? …” or “… invoice … its number.”
invented_period	a year / date the user never wrote	“Which year do you mean?”

invented_target only checks the name / id, not the new values, and it works on substrings (the user wrote “Desk Lamp”, the bot wrote “Lamp” → not caught).

S9 Create an invoice step by step

The user	Types “make an invoice for Karl Becker”, then answers three forms: dates, products, paid/unpaid; then clicks Yes.
The path	The invoice bot asks in a fixed order; each question opens a form · the forms' answers come back as chat messages to the waiting invoice bot · the final CREATE_INVOICE uses names; Python turns them into numbers, fills in prices and builds the proposal.



Step by step

#	From → to	What happens
		Turn 1
1	Browser → api.py	**POST /chat “make an invoice for Karl Becker”**
2	router.py → Ollama	**route() → Route: invoice**
3	router.py → bot.py	**bot.converse(inv-invoice-bot)**
4	bot.py → Ollama	**chat(invoice bot)**

#	From → to	What happens
5	Ollama → bot.py	**“What are the invoice date and the due date?”** Modelfile-invoice fixes the order and the exact questions: 1 dates, 2 lines, 3 customer (skipped – Karl Becker was named), 4 status.
6	bot.py → router.py	**{reply, open: true}**
7	router.py	**invoice + open: “date” → ui ["dates"]**
8	router.py → Browser	**reply + date picker** Invoice date not before today; due date not before the invoice date.
		Turn 2: “invoice date 21/10/2026, due date 20/11/2026” → bot: “Which products and quantities?” → ui lines (GET /products for the list)
		Turn 3: “products: 2 Desk Lamp, 1 USB Cable 1m” → bot: “Is the invoice paid or unpaid?” → ui status (two buttons)
		Turn 4
9	Browser → api.py	**POST /chat “unpaid”**
10	router.py	**waiting → invoice (no routing)**
11	bot.py → Ollama	**chat(invoice bot, 7 messages)**
12	Ollama → bot.py	**CREATE_INVOICE {customer, date, due_date, status, lines: [{product, …}]}**
13	bot.py	**guards (CREATE: no target check)**
14	bot.py → invoices.py	**check_command(CREATE_INVOICE, data)** resolve(): find_customers("Karl Becker") and find_products(…) per line; exactly one match each, else “There is no customer called …” / “There are 2 products called …”. Then check_invoice on the resolved data.
15	invoices.py → storage.py	**load customers, products** check_invoice: customer exists, both dates valid DD/MM/YYYY, due ≥ date, status paid/unpaid, ≥ 1 line, each product exists, quantity a whole number > 0, a given unit_price ≥ 0.
16	bot.py → invoices.py	**build_proposal(…)** resolve again; make_lines puts the product's sell_price where no unit_price was given; summary lists each line and the total.
17	router.py → Browser	**{proposal: POST /invoices, body with ids}**
18	Browser → api.py	**Yes → POST /invoices** api.add_invoice: InvoiceIn → check_invoice again → create_invoice.
19	invoices.py → storage.py	**next_id → INV-0nn; make_lines; save("invoices")** Lines get numbers 1, 2, …; save rewrites the invoices and invoice_lines tables.
20	api.py → Browser	**201 {…, lines, total} → “Saved as INV-0nn.”**

Names in, numbers out

What the model writes (names, as the user said them)

```
CREATE_INVOICE {"customer": "Karl Becker", "date": "21/10/2026", "due_date": "20/11/2026",
                "status": "unpaid",
                "lines": [{"product": "Desk Lamp", "quantity": 2}, {"product": "USB Cable 1m", "quantity": 1}]}
```

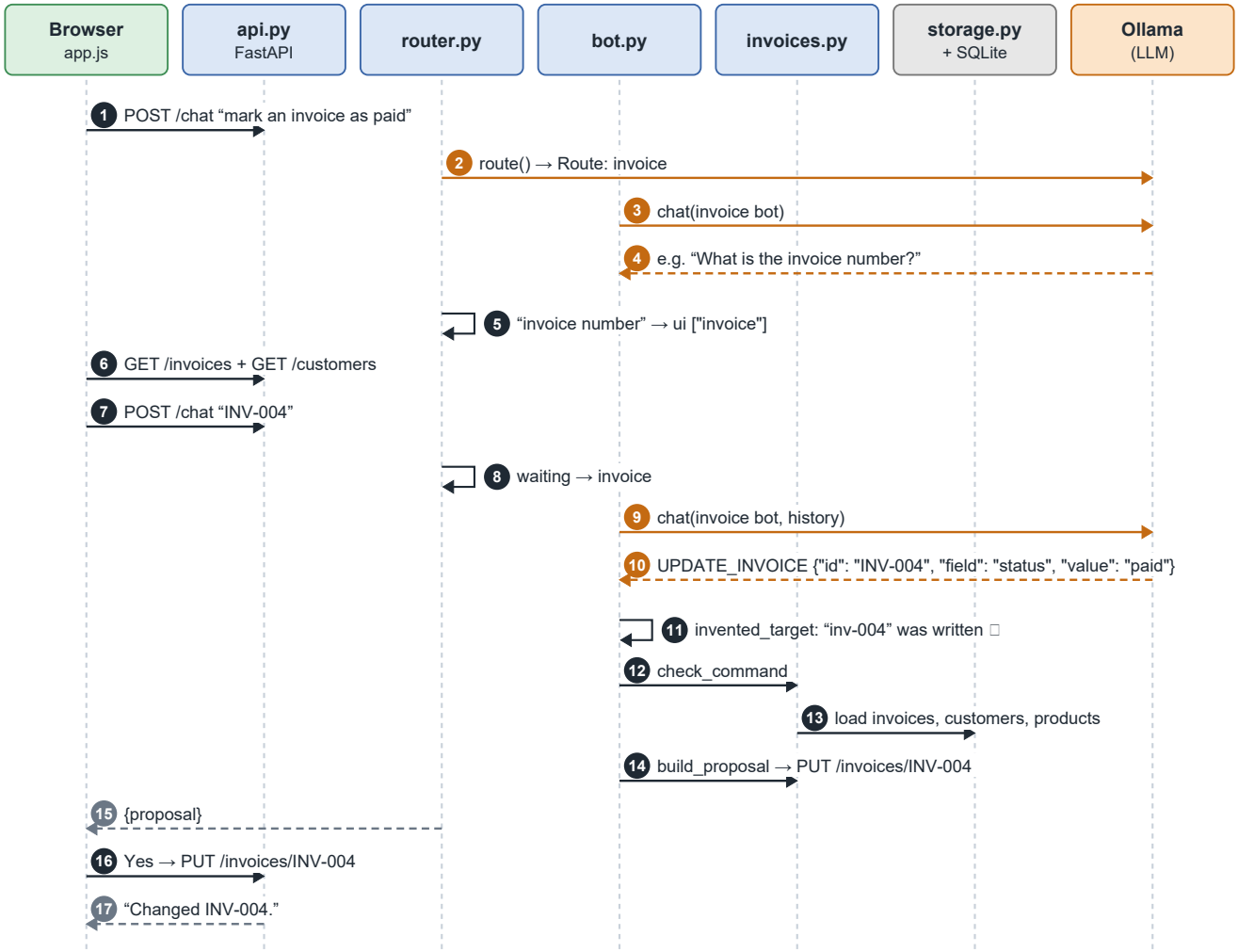
What Python proposes (numbers and prices from the database)

```
{"method": "POST", "path": "/invoices",
 "body": {"customer_id": "CUS-002", "date": "21/10/2026", "due_date": "20/11/2026", "status": "unpaid",
          "lines": [{"product_id": "PRD-003", "quantity": 2, "unit_price": 29.9},
                    {"product_id": "PRD-001", "quantity": 1, "unit_price": 4.5}]},
 "summary": "Create invoice for CUS-002 (Karl Becker) | date 21/10/2026 | due 20/11/2026 | unpaid |
             1: 2 x Desk Lamp x 29.90 = 59.80; 2: 1 x USB Cable 1m x 4.50 = 4.50 | total 64.30"}
```

If any step fails (e.g. a product name matches nothing), Python's error is the reply, it is open, and for the invoice bot the router may open a chooser because the error text contains “product” or “customer”.

S10 Mark an invoice as paid

The user	Types “mark an invoice as paid”, picks INV-004 in the chooser, clicks Yes.
The path	invoice bot asks for the number → invoice chooser → bare “INV-004” → UPDATE_INVOICE status paid → Python builds the full invoice with one field changed → PUT after Yes. From now on the invoice counts as turnover.



Step by step

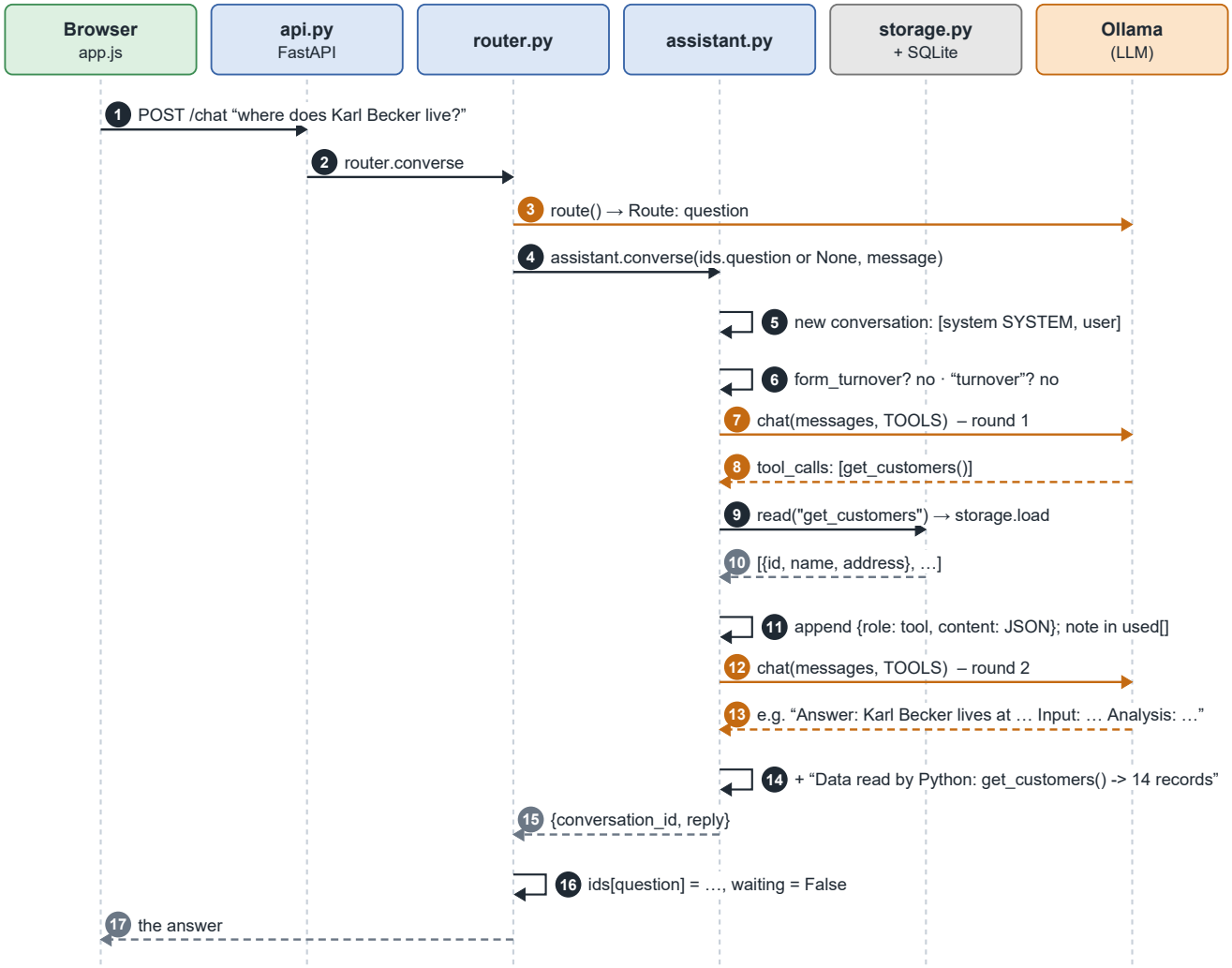
#	From → to	What happens
1	Browser → api.py	**POST /chat “mark an invoice as paid”**
2	router.py → Ollama	**route() → Route: invoice**
3	bot.py → Ollama	**chat(invoice bot)**
4	Ollama → bot.py	**e.g. “What is the invoice number?”** Modelfile rule: an existing invoice is named by its number; no number → ask.
5	router.py	**“invoice number” → ui ["invoice"]**
6	Browser → api.py	**GET /invoices + GET /customers** showInvoicePicker: newest first, each entry “customer | status | total”.
7	Browser → api.py	**POST /chat “INV-004”**
8	router.py	**waiting → invoice**
9	bot.py → Ollama	**chat(invoice bot, history)**
10	Ollama → bot.py	**UPDATE_INVOICE {"id": "INV-004", "field": "status", "value": "paid"}**
11	bot.py	**invented_target: “inv-004” was written □**

#	From → to	What happens
12	bot.py → invoices.py	**check_command** Invoice exists; field is date, due_date or status; changed_invoice() = the stored invoice with the one field replaced → check_invoice on the whole thing.
13	invoices.py → storage.py	**load invoices, customers, products**
14	bot.py → invoices.py	**build_proposal → PUT /invoices/INV-004** body = all fields (customer_id, date, due_date, status, lines) because PUT replaces the whole invoice. summary “Change status of INV-004: 'unpaid' -> 'paid'”.
15	router.py → Browser	**{proposal}**
16	Browser → api.py	**Yes → PUT /invoices/INV-004** api.change_invoice: 404 if missing → check_invoice → update_invoice → save.
17	api.py → Browser	**“Changed INV-004.”**

> Turnover counts only invoices with status paid, by their due date. So this one click moves money into the turnover reports of the invoice's due-date month (S13, S14).

S11 Ask a question: where does a customer live?

The user	Types “where does Karl Becker live?”.
The path	router model → question · assistant model calls get_customers · Python returns all customers as JSON · the model reads the address out of it and answers in three parts · Python adds which data it read.



Step by step

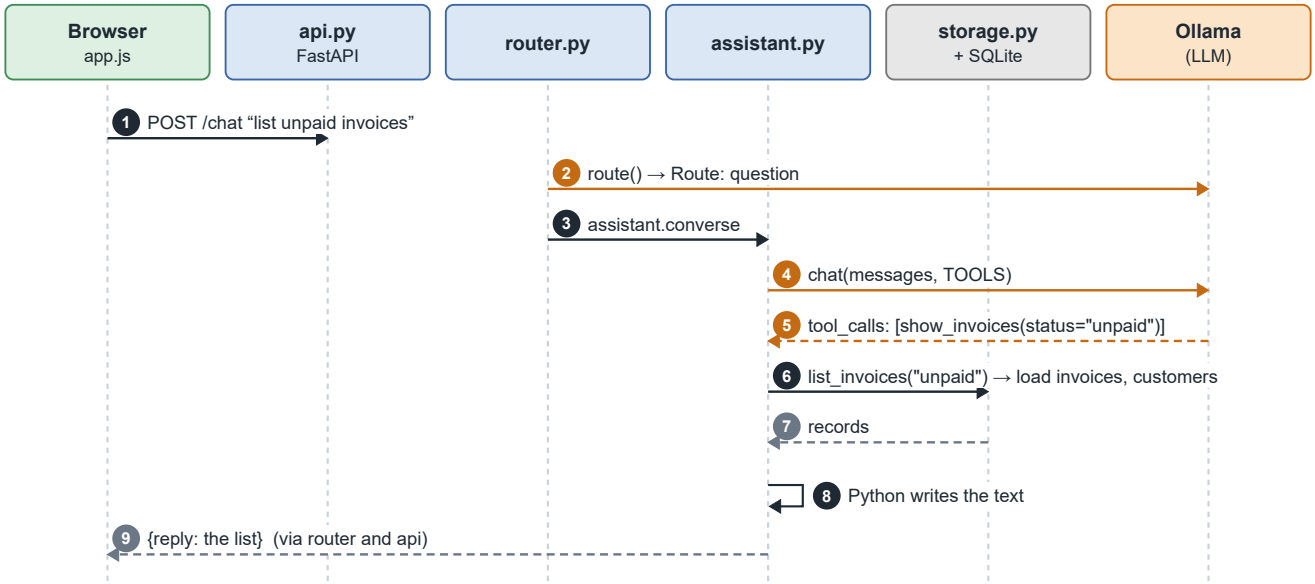
#	From → to	What happens
1	Browser → api.py	**POST /chat “where does Karl Becker live?”**
2	api.py → router.py	**router.converse**
3	router.py → Ollama	**route() → Route: question** Router rule: names a customer but only reads → question.
4	router.py → assistant.py	**assistant.converse(ids.question or None, message)**
5	assistant.py	**new conversation: [system SYSTEM, user]** The conversation list comes from bot.get_conversation, the same store the change bots use.
6	assistant.py	**form_turnover? no · “turnover”? no**
7	assistant.py → Ollama	**chat(messages, TOOLS) – round 1** llama3.2:3b, temperature 0, with the 19 tool definitions.
8	Ollama → assistant.py	**tool_calls: [get_customers()]**
9	assistant.py → storage.py	**read("get_customers") → storage.load** If the model had passed a filter, Python would still return all records plus a note.
10	storage.py → assistant.py	**[{id, name, address}, …]**

#	From → to	What happens
11	assistant.py	**append {role: tool, content: JSON}; note in used[]**
12	assistant.py → Ollama	**chat(messages, TOOLS) – round 2**
13	Ollama → assistant.py	**e.g. "Answer: Karl Becker lives at … Input: … Analysis: …"** No tool calls → the loop ends.
14	assistant.py	**+ "Data read by Python: get_customers() -> 14 records"**
15	assistant.py → router.py	**{conversation_id, reply}**
16	router.py	**ids[question] = …, waiting = False**
17	router.py → Browser	**the answer**

> Here – and only in this kind of answer – the content comes from the model: it picks the right record out of the JSON. Python does not check that the address it repeats is correct. That is why the reply ends with the list of tools that ran, so the user can see what the answer was based on.

S12 List unpaid invoices / is one invoice paid?

The user	Types “list unpaid invoices”, later “is INV-011 paid?”.
The path	The assistant model only chooses the tool show_invoices and its filter · Python writes the list and returns it at once · the model never sees the result, so it cannot copy it wrongly.



Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “list unpaid invoices”**
2	router.py → Ollama	**route() → Route: question**
3	router.py → assistant.py	**assistant.converse**
4	assistant.py → Ollama	**chat(messages, TOOLS)** SYSTEM teaches: open, pending, still due = unpaid; settled, cleared = paid.
5	Ollama → assistant.py	**tool_calls: [show_invoices(status="unpaid")]**
6	assistant.py → storage.py	**list_invoices("unpaid") → load invoices, customers** A status other than paid / unpaid ("null", "all") means all invoices.
7	storage.py → assistant.py	**records**
8	assistant.py	**Python writes the text** One line per invoice and a count line. Put into the history as tool result and as assistant message, so a follow-up question sees what the user saw.
9	assistant.py → Browser	**{reply: the list} (via router and api)** Returned at once: no second model round, no “Answer / Input / Analysis”.

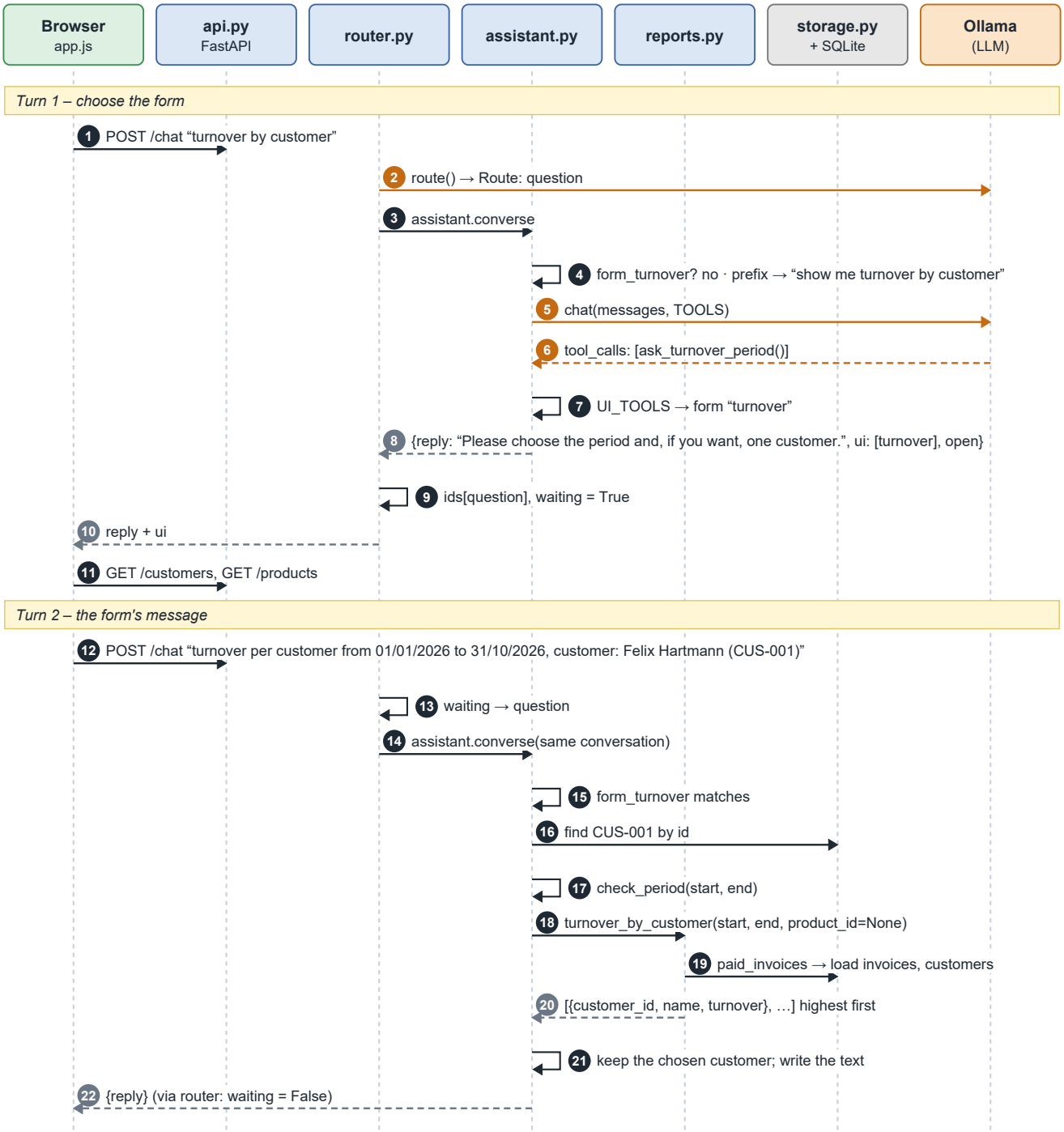
The reply (format from list_invoices)

```
INV-003 | Karl Becker | 01/09/2026 | due 01/10/2026 | unpaid | 120.00
INV-007 | Lena Berg | 12/10/2026 | due 11/11/2026 | unpaid | 59.80
2 unpaid invoices.
```

Question	Tool call	Python's answer
“is INV-011 paid?”	show_invoices(invoice_id="INV-011")	the one line with its status; a status filter the model adds is ignored
“is INV-099 paid?” (no such invoice)	show_invoices(invoice_id="INV-099")	“There is no invoice INV-099.”
“how many invoices are unpaid?”	get_invoices(status="unpaid") (a read tool)	{count, records} goes back to the model; it copies the count into its answer

S13 Turnover per customer

The user	Types “turnover by customer”. A form appears; the user picks a period and, optionally, one customer and/or one product, and clicks “Show turnover”.
The path	Turn 1: the assistant model only has to pick the right form · turn 2: the form's message is recognised by form_turnover and answered by Python with reports.py – no model at all.



Step by step

#	From → to	What happens
		Turn 1 – choose the form
1	Browser → api.py	**POST /chat “turnover by customer”**
2	router.py → Ollama	**route() → Route: question**
3	router.py → assistant.py	**assistant.converse**

#	From → to	What happens
4	assistant.py	**form_turnover? no · prefix → “show me turnover by customer”**
5	assistant.py → Ollama	**chat(messages, TOOLS)**
6	Ollama → assistant.py	**tool_calls: [ask_turnover_period()]** If the model calls get_turnover_by_customer instead, FORM_FOR swaps it for ask_turnover_period.
7	assistant.py	**UI_TOOLS → form “turnover”** History: tool “The form is shown to the user. Wait for the answer.” + assistant question.
8	assistant.py → router.py	**{reply: “Please choose the period and, if you want, one customer.”, ui: [turnover], open}**
9	router.py	**ids[question], waiting = True**
10	router.py → Browser	**reply + ui**
11	Browser → api.py	**GET /customers, GET /products** showTurnoverPicker("customer"): From, To, Customer (empty = all), Product (empty = all). Entries are “name (id)”.
		Turn 2 – the form's message
12	Browser → api.py	**POST /chat “turnover per customer from 01/01/2026 to 31/10/2026, customer: Felix Hartmann (CUS-001)”**
13	router.py	**waiting → question**
14	router.py → assistant.py	**assistant.converse(same conversation)**
15	assistant.py	**form_turnover matches** Regex reads kind, start, end and the chosen “customer: … (id)” / “product: … (id)”. Nothing chosen = all.
16	assistant.py → storage.py	**find CUS-001 by id** Unknown id → “There is no customer CUS-001.” – never silently “all”.
17	assistant.py	**check_period(start, end)** Both dates valid DD/MM/YYYY, end not before start; else that message is the answer.
18	assistant.py → reports.py	**turnover_by_customer(start, end, product_id=None)**
19	reports.py → storage.py	**paid_invoices → load invoices, customers** Keeps invoices with status paid and due_date between start and end (both included). Sums quantity × unit_price per customer (with a product: only that product's lines).
20	reports.py → assistant.py	**[{customer_id, name, turnover}, …] highest first**
21	assistant.py	**keep the chosen customer; write the text**
22	assistant.py → Browser	**{reply} (via router: waiting = False)**

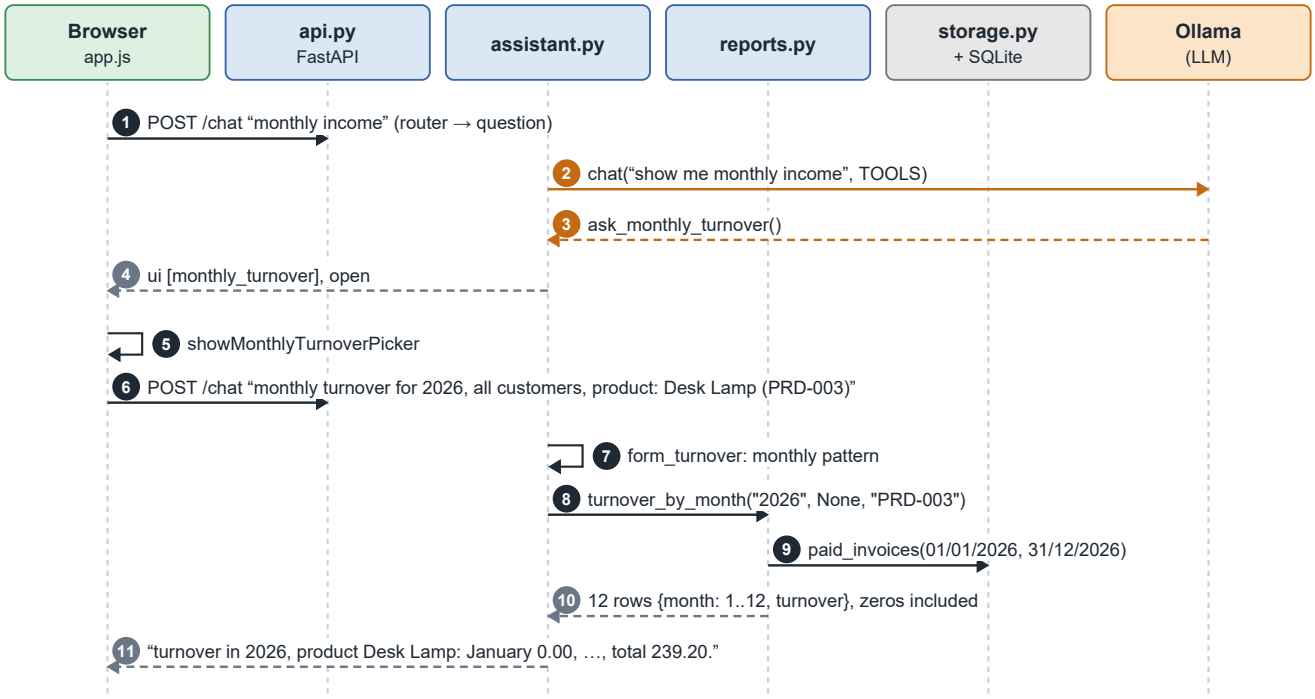
Python's answers (format from form_turnover; amounts are examples)

```
from 01/01/2026 to 31/10/2026, customer Felix Hartmann: Felix Hartmann 1700.00, total 1700.00.
from 01/01/2026 to 31/10/2026: Felix Hartmann 1700.00, Karl Becker 120.00, total 1820.00.        (all customers)
from 01/01/2026 to 31/10/2026, product Desk Lamp: no turnover, total 0.00.
```

Customers without turnover in the period are left out of the list; a chosen customer without turnover gives “no turnover, total 0.00”.

S14 Monthly turnover and turnover per product

The user	Types “monthly income” (or “revenue per product”), fills the form, clicks “Show turnover”.
The path	Same two turns as S13; only the form and the report function differ.



Step by step

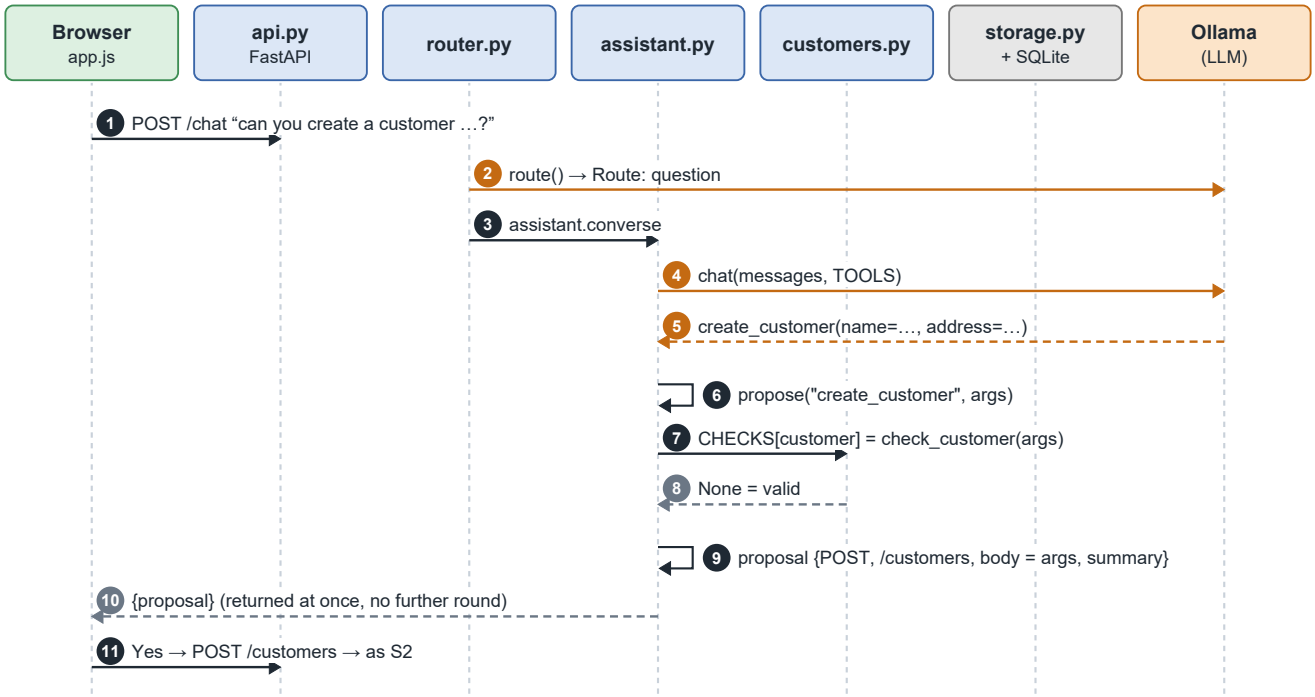
#	From → to	What happens
1	Browser → api.py	**POST /chat “monthly income” (router → question)**
2	assistant.py → Ollama	**chat(“show me monthly income”, TOOLS)**
3	Ollama → assistant.py	**ask_monthly_turnover()**
4	assistant.py → Browser	**ui [monthly_turnover], open**
5	Browser	**showMonthlyTurnoverPicker** Year (4 digits, this year preset), customer and product optional, “name (id)”.
6	Browser → api.py	**POST /chat “monthly turnover for 2026, all customers, product: Desk Lamp (PRD-003)”**
7	assistant.py	**form_turnover: monthly pattern**
8	assistant.py → reports.py	**turnover_by_month("2026", None, "PRD-003")**
9	reports.py → storage.py	**paid_invoices(01/01/2026, 31/12/2026)** Each paid invoice goes into the month of its due date; with a product only its lines count.
10	reports.py → assistant.py	**12 rows {month: 1..12, turnover}, zeros included**
11	assistant.py → Browser	**“turnover in 2026, product Desk Lamp: January 0.00, …, total 239.20.”**

Form	Message pattern	reports.py function	Filters
turnover	turnover per customer from A to B, …	turnover_by_customer(start, end, product_id)	one customer (kept afterwards), one product (lines)
product_turnover	turnover per product from A to B, …	turnover_by_product(start, end, customer_id)	one product (kept afterwards), one customer (invoices)
monthly_turnover	monthly turnover for YYYY, …	turnover_by_month(year, customer_id, product_id)	both, all 12 months shown

The same functions serve GET /reports/customers, /reports/products and /reports/months (S19). The website does not use those endpoints; it always goes through the chat and form_turnover.

S15 A change through the assistant

The user	Types a change phrased as a question, e.g. “can you create a customer Lena Berg, Ringstr. 2, 44135 Dortmund?”, and the router sends it to the assistant.
The path	The assistant has write tools too · propose() checks the arguments and returns a proposal · the Yes path is exactly the same as for the change bots.



Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “can you create a customer …?”**
2	router.py → Ollama	**route() → Route: question**
3	router.py → assistant.py	**assistant.converse**
4	assistant.py → Ollama	**chat(messages, TOOLS)**
5	Ollama → assistant.py	**create_customer(name=…, address=…)**
6	assistant.py	**propose("create_customer", args)** Splits the tool name into action + entity. For update / delete it takes customer_id / product_id / invoice_id out and the record must exist.
7	assistant.py → customers.py	**CHECKS[customer] = check_customer(args)**
8	customers.py → assistant.py	**None = valid**
9	assistant.py	**proposal {POST, /customers, body = args, summary}** Tool message for the model: “Proposal shown to the user. Nothing is saved until the user clicks Yes.”
10	assistant.py → Browser	**{proposal} (returned at once, no further round)**
11	Browser → api.py	**Yes → POST /customers → as S2**

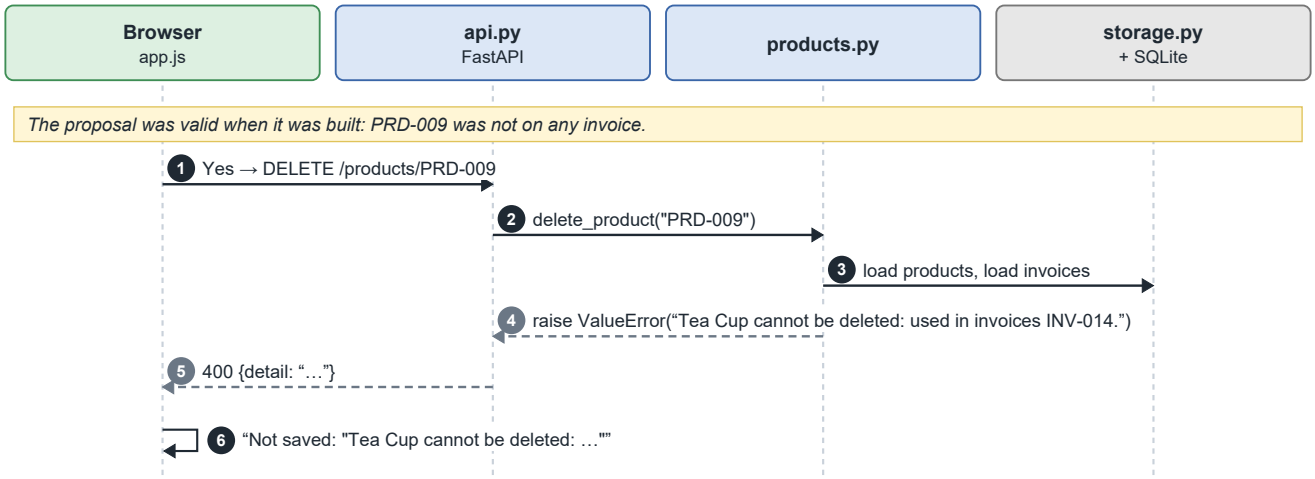
Two ways to a proposal

	Change bot (bot.converse)	Assistant (propose)
Model writes	a command line with names	a tool call with ids
Record found by	name, exact first, else contains	id (the model reads it from get_customers first)
Delete checks before Yes	used on an invoice / paid → refused	only: does it exist (open_problems #22) – the WRITE endpoint refuses later

	Change bot (bot.converse)	Assistant (propose)
Summary	readable: “Create customer: Lena Berg | …”	JSON: “Create customer: {"name": …}”
After Yes	the same WRITE endpoints	the same WRITE endpoints

S16 Yes, but the data changed in the meantime

The user	Gets the proposal “Delete product PRD-009 …”, but before clicking Yes an invoice with this product is created (another tab or program). Then clicks Yes.
The path	The proposal is only a description; the WRITE endpoint checks everything again against the database as it is now and refuses.



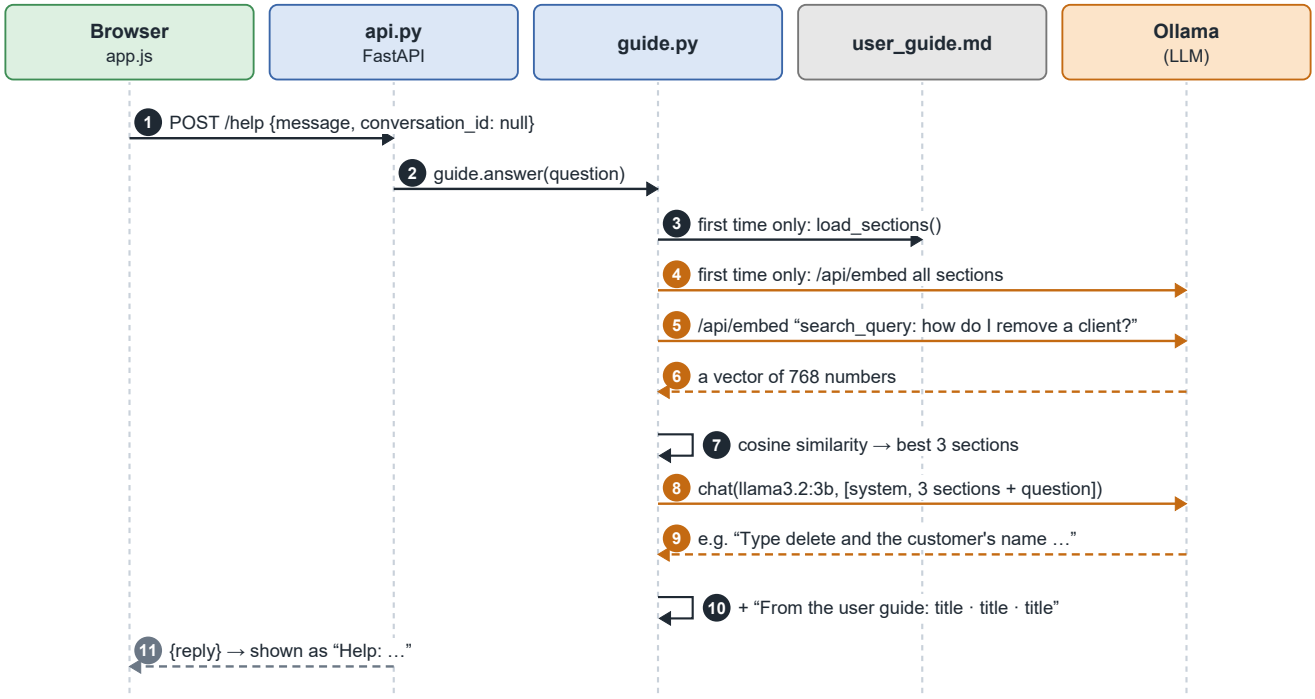
Step by step

#	From → to	What happens
		The proposal was valid when it was built: PRD-009 was not on any invoice.
1	Browser → api.py	**Yes → DELETE /products/PRD-009**
2	api.py → products.py	**delete_product("PRD-009")**
3	products.py → storage.py	**load products, load invoices** The new invoice now has a line with PRD-009.
4	products.py → api.py	**raise ValueError(“Tea Cup cannot be deleted: used in invoices INV-014.”)**
5	api.py → Browser	**400 {detail: “…”}**
6	Browser	**“Not saved: "Tea Cup cannot be deleted: …"”** Nothing changed. The table is not reloaded.

What changed meanwhile	Endpoint	Answer
Record deleted	PUT / DELETE	404 “… not found”
Product now on an invoice	DELETE /products/…	400 “… used in invoices …”
Invoice now paid	DELETE /invoices/…	400 “… cannot be deleted: it is paid.”
Customer of a new invoice deleted	POST /invoices	400 “There is no customer …”

S17 The Help button

The user	Types “how do I remove a client?” and clicks Help (an empty field asks “How do I use the chat?”).
The path	A separate path: no router, no session, no database · guide.py finds the 3 sections of the user guide closest in meaning (embeddings) and lets a model answer only from them.



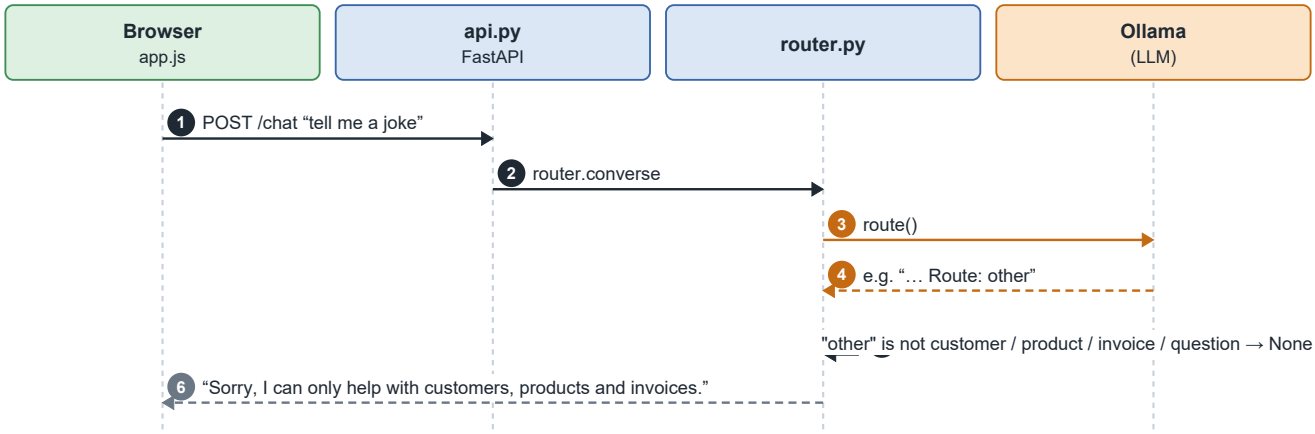
Step by step

#	From → to	What happens
1	Browser → api.py	**POST /help {message, conversation_id: null}** askHelp() – the Help button never uses /chat.
2	api.py → guide.py	**guide.answer(question)**
3	guide.py → user_guide.md	**first time only: load_sections()** Reads docs/user_guide.md, removes the author comment, cuts it at every “## ” heading → 21 sections {title, text}.
4	guide.py → Ollama	**first time only: /api/embed all sections** nomic-embed-text, each text prefixed “search_document: ”. The vectors stay in SECTIONS until the server restarts.
5	guide.py → Ollama	**/api/embed “search_query: how do I remove a client?”**
6	Ollama → guide.py	**a vector of 768 numbers**
7	guide.py	**cosine similarity → best 3 sections** Similar meaning → similar vectors, so “remove a client” finds “How do I delete a customer?”.
8	guide.py → Ollama	**chat(llama3.2:3b, [system, 3 sections + question])** Rules: answer only from the sections, treat synonyms as the same, anything not described → “I cannot help with that.”
9	Ollama → guide.py	**e.g. “Type delete and the customer's name …”**
10	guide.py	**+ “From the user guide: title · title · title”**
11	api.py → Browser	**{reply} → shown as “Help: …”**

After editing docs/user_guide.md the server must be restarted: the sections and their vectors are built once and kept in memory.

S18 A message that is not about invoicing

The user	Types “tell me a joke”.
The path	The router model answers with route “other”, which is not a known bot · router.py answers itself.



Step by step

#	From → to	What happens
1	Browser → api.py	**POST /chat “tell me a joke”**
2	api.py → router.py	**router.converse**
3	router.py → Ollama	**route()**
4	Ollama → router.py	**e.g. “… Route: other”**
5	router.py	**"other" is not customer / product / invoice / question → None**
6	router.py → Browser	**“Sorry, I can only help with customers, products and invoices.”** No entity in the answer, so the browser shows “Bot:” instead of “Bot (customer):”.

A change bot can also refuse by itself, e.g. the customer bot with “Sorry, I can only help with customers.” bot.converse marks exactly this reply as **not open**, so the router stops waiting and the next message is routed fresh. While a bot *is* waiting (after a question or an error), an off-topic message is not routed at all – it goes to the waiting bot, which then usually refuses it this way.

S19 Other entry points

The user	Another program, a test, or a developer in http://127.0.0.1:8000/docs.
The path	The same modules, entered at a different point.

Endpoint	Goes to	Difference to the website's path
POST /customers/chat, /products/chat, /invoices/chat	bot.converse with that entity's functions	No router: no routing, no NEGATIVE_PRICE check, no ui keys. The conversation_id is the bot's own id.
POST /assistant/chat	assistant.converse	No router, no ui merging; the assistant's own ui keys still come back.
GET /reports/customers?start&end;, /reports/products?start&end;	reports.check_period, then turnover_by_customer / _by_product	Plain JSON rows; 400 for an invalid period. Not used by the website.
GET /reports/months?year&customer;_id&product;_id	reports.turnover_by_month	400 invalid year, 404 unknown customer / product.
GET/POST/PUT/DELETE /customers, /products, /invoices	entity modules	The WRITE endpoints of section 2.5; no Yes needed – the caller is trusted to have asked.

The tests in tests/ use a further entry point: they call router.converse(conversation_id, message) in-process – the same path as POST /chat, without HTTP – on a copy of the database.

Appendix A Where every piece of state lives

State	Where	Written by	Read by	Survives
conversationId	app.js variable	every /chat answer	sendMessage	until page reload
SESSIONS	router.py	router.converse	router.converse	until server restart
CONVERSATIONS	bot.py	bot.converse, assistant.converse (via get_conversation)	the same, to build the next request to Ollama	until server restart
SECTIONS (+ vectors)	guide.py	build_index (first help question)	search	until server restart
Bot rules	modelfiles/* → Ollama models	ollama create	Ollama on every call	until rebuilt
Assistant rules + tools	assistant.SYSTEM, TOOLS	the code	assistant.chat	code
Records	data/invoices.db	storage.save (only from create_/update_/delete_…)	storage.load	permanently

A.2 Which parts may change the database

Only nine functions call storage.save: create_customer, update_customer, delete_customer, create_product, update_product, delete_product, create_invoice, update_invoice, delete_invoice – and only the WRITE endpoints in api.py call them. No chat function (router, bot, assistant, guide) calls any of them. This is what “the chat never saves” means in the code.

Appendix B Function index

Each function that appears in the scenarios, where it lives and where to see it in action.

Function	File	Job (short)	Scenarios
sendMessage, showProposal, confirmProposal	static/app.js	send to /chat; Yes/No; send the proposal	S2–S16
loadList, switchTab	static/app.js	tables	S1
show…Picker / show…Form	static/app.js	the forms (2.6)	S3, S4, S7–S10, S13, S14
askHelp	static/app.js	Help button	S17
chat, customer_chat, …, help_chat	app/api.py	chat endpoints	all chat scenarios, S19
add_/change_/remove_…	app/api.py	WRITE endpoints	S2, S4, S9, S10, S16
router.converse, route	app/chat/router.py	pick bot, waiting, ui	S2–S15, S18
bot.chat	app/chat/bot.py	one Ollama chat request	all model calls except the assistant's
bot.converse	app/chat/bot.py	command + guards + check + proposal	S2–S10
parse_command, clean_command, invented_target, invented_period	app/chat/bot.py	guards	S8
assistant.converse, chat	app/chat/assistant.py	tool loop	S11–S15
read, list_invoices	app/chat/assistant.py	read tools, invoice list text	S11, S12
form_turnover	app/chat/assistant.py	turnover form messages → answer	S13, S14
propose	app/chat/assistant.py	write tool → proposal	S15
guide.answer, search, build_index, embed	app/chat/guide.py	Help (RAG)	S17
check_command, build_proposal, answer	app/entities/*.py	the three functions bot.converse gets	S2–S10
check_customer / check_product / check_invoice	app/entities/*.py	value checks (chat and WRITE)	S2, S4, S9, S10, S15, S16
find_customers / find_products	app/entities/*.py	lookup by name, exact first	S4, S6, S7, S9
resolve, make_lines, changed_invoice	app/entities/invoices.py	names → ids, prices, one field changed	S9, S10
invoice_total, to_date	app/entities/invoices.py	total; DD/MM/YYYY → date	S1, S9, S12–S14
paid_invoices, turnover_by_…, check_period	app/core/reports.py	turnover R1–R3	S13, S14, S19
load, save, next_id, find	app/core/storage.py	SQLite	everywhere

B.2 Words used in this document

Word	Meaning here
Bot / model	A language model running in Ollama. “Change bot” = the customer, product or invoice bot; “assistant” = the question bot with tools; “router” = the bot that only sorts.
Command line	Text like CREATE_CUSTOMER {…} that a change bot writes; Python reads it.
Tool call	The assistant model's way to ask Python to run a function (get_customers, …).
Proposal	{method, path, body, summary}: a REST call described but not made. The browser makes it after Yes.
open / waiting	The bot's task is not finished (it asked a question or got an error); the next message goes back to it without routing.
ui	A list of form names in a chat answer; the browser shows those forms.
Turnover	Sum of quantity × unit_price over the lines of paid invoices whose due date is in the period.